

赤外線マスク同期を用いた 複数 Arduino による輪唱演奏システム

情報工学科 2 年

25G1046

櫛濱 和輝

—— チーム 9 ——

久家 力孔 (24G1052) 手代木 巧 (24G1098)

渡邊 乃斗 (24G1145) 家田 祐希 (25G1010)

木下 遼介 (25G1043)

2026 年 7 月 16 日

1 はじめに

近年、IoT 技術の普及によって、複数のデバイスがネットワークを介してリアルタイムに連携するシステムが様々な分野で実用化されている [16]. IoT 技術やプログラミング教育の重要性が高まるなか [12], STEAM 教育やものづくりを通じた実践的学習の有効性が示されており [14, 15], 大学教育においても PBL やハッカソン形式の授業が学生の主体性向上に寄与することが報告されている [21, 26, 13]. IoT を題材としたチーム開発型 PBL の効果も実証されており [25], ハッカソンを人材育成に活用する事例 [27] やオンラインでの実践報告 [22] も増えている. さらに, IoT ロボットを用いた協調学習 [23] やプログラミング教育を通じた多世代間の教え合いの可能性 [24] も示されている. 製造業における機械の協調制御やスマートホームの機器連携など, 複数の自律した機器を同期させる技術は現代の組み込みシステムに広く求められている. 合奏はこの同期という課題が音として直接聞こえる題材である. Rasch によると, 合奏において奏者間の発音タイミングのずれはおおよそ 30~50 ms の範囲に収まっており, これを大きく超えるずれは聴き手に同期の乱れとして知覚される [1]. つまり, 複数の機器で合奏を成立させるには, 数十 ms 以内の精度で状態を共有し続ける必要がある.

電子楽器の同期に関する既存技術としては, MIDI や Wi-Fi・Bluetooth を用いたシステムが挙げられる. MIDI は音高やタイミング, 強さといった精密な情報を伝達できる規格である [2] が, 専用のインターフェースと対応ライブラリが必要であり, 同期のきっかけだけを伝えたい本プロジェクトの用途には過剰な仕様となる. Wi-Fi や Bluetooth を用いた合奏システムは遅延補正の技術も発達しているものの, 通信インフラの品質に動作が左右されるため, 多数の機器が電波を発する実験室環境では干渉や接続不安定のリスクが残る. そこで本プロジェクトでは, 伝達する情報を「どの楽器がいま演奏に参加しているか」という最小限の同期情報に限定し, 赤外線通信で実現する方針を採った. 赤外線はネットワーク環境を必要とせず, LED と受光モジュールだけで構成できるため, 小規模な組み込みシステムに低コストで導入できる [3].

本プロジェクトでは, 5 台の Arduino を用いたオーケストラシステム「赤外線通信で歌うカエル」を制作した. 1 台を指揮側の親機, 4 台をピアノ・フルート・トランペット・ドラムを担当する演奏側の子機とし, 楽曲「かえるのうた」を輪唱形式で演奏する. 親機は演奏中の楽器集合を表すマスクデータを赤外線で全子機へ一斉送信し, 各子機は自分の担当ビットが有効なときにのみ譜面を 1 コマ進め, PC 上の Processing へ音符情報を送って楽器音を再生させる. 演奏者はボタン操作で各楽器を任意のタイミングで参加・退場させることができ, 可変抵抗によるテンポの連続調整にも対応する. さらに子機に接続した LCD には, かえるの口パクアニメーションと歌詞を表示する.

報告者は本システムのうち, ピアノ音源の合成と Processing 側の再生処理, ピアノ担当子機と Arduino との通信用関数, ドラム担当子機のプログラム, ブレッドボードの設計, およびテンポ管理プログラムの実装を担当した. テンポ管理は渡邊との共同担当である. また, 開発終盤には親機と子機で楽譜 1 周分の tick 数が食い違い特定の音が周期的に抜けるバグの修正と, 休符が休符として聞こえない問題の修正を行った.

評価では、単体テストで各楽器音色と通信機能を確認したのち、第9週末に親機1台と子機4台による輪唱の実機テストを実施し、ボタン押下順に楽器が参加するカノン演奏が音抜けや休符抜けなく成立することを確認した。演奏タイミング誤差と赤外線通信成功率の定量計測は、計測用の同期パルス出力とログ機構を実装済みである。本報告書では、システム全体の構成と報告者の担当部分の実装を述べたうえで、実機テストの結果と残された課題について考察する。

2 作成したシステムの概要

本システムは、同期制御機能、楽曲制御機能、およびLCD表示機能から構成される。同期制御は親機からの赤外線信号の送受信、楽曲制御は音符情報の生成・演奏タイミング制御・音声出力、LCD表示はかえるのアニメーションと歌詞のスクロール表示をそれぞれ担う。まずシステム全体の構成を説明する。

2.1 システム構成

システムの構成を図1に示す。本システムは、指揮側の親機Arduino1台、演奏側の子機Arduino4台、および楽器音を再生するPC4台から構成される。いずれのArduinoもArduino Uno R4を用いた。ワンボードマイコンは教育現場でのプログラミング学習に広く活用されており[17, 18]、無線通信を用いた教材開発の事例も報告されている[19]。親機には赤外線LED、演奏開始・リセット・子機参加用のタクトスイッチ、およびテンポ調整用の可変抵抗を接続する。各子機には赤外線受光モジュールOSRB38C9AA[8]を接続し、USBシリアルでPCと接続する。PC側ではProcessingが動作し、子機から届いた音符情報をもとに楽器音を合成してスピーカから出力する。楽器音の生成にはProcessingのサウンドライブラリであるMinim[6]を用いた。

システム全体のデータの流れは一方向である。親機は現在のtick番号と参加楽器マスクを赤外線でブロードキャストし、子機はそれを受けて自分の譜面配列を参照し、該当する音符情報をProcessingへ送信する。Processingは受信した音符情報から楽器音を合成して再生する。子機はLCDへの表示出力も並行して行う。

2.2 製品分解構造 (PBS)

本システムの製品分解構造 (PBS) を図2に示す。システムはProcessing (PC側)、Arduino親機、Arduino子機の3つのサブシステムから構成される。

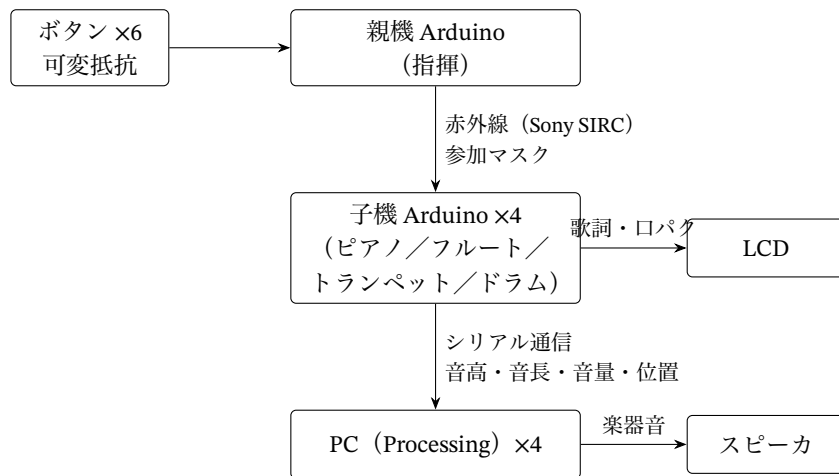
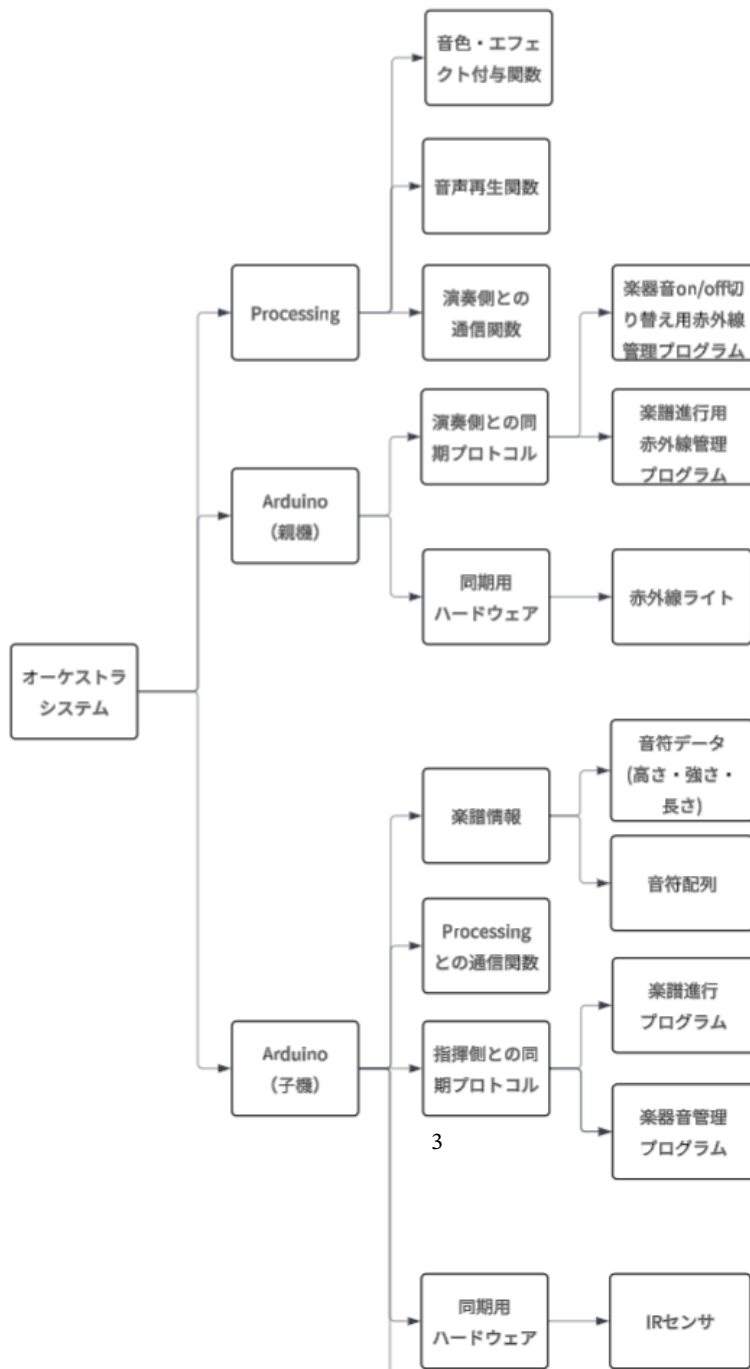


図 1 システム構成とデータの流れ



2.3 赤外線同期プロトコル

親機と子機間の同期には、Sony SIRC 12 ビットフォーマット [4] を用いた。送受信には Arduino 用ライブラリの IRremote[5] を利用している。赤外線リモコンプロトコルとしては NEC フォーマット [3] (固定 32 ビット) が広く知られるが、NEC はアドレスとコマンドの各 8 ビットに対する反転バイトをノイズ検出用に付加するため、フレーム長が大きい。本システムで実際に送信するデータは楽器マスクの 4 ビットのみであり、通信遅延を目標の 30 ms 以内に抑えるために、ビット幅を 12/15/20 ビットから選択できる Sony SIRC の最短 12 ビット (コマンド 7 ビット + アドレス 5 ビット) を採用した。SIRC には NEC のような反転バイトによるエラー検出がないが、屋内環境では目視レベルの赤外線ノイズが少なく、10000 回の受信テストで全て成功していることから実用上の問題はない。加えて、赤外線は室内に人数がいても、送受信経路上に人体や機材などの部分的な遮蔽物がある状態でも受信できることが確認された。

コマンド部 7 ビットのうち下位 4 ビットを演奏中の楽器集合を表すマスクとして使用し、上位 3 ビットは常に 0 とする。各ビットと子機の対応を表 1 に示す。アドレス部 5 ビットは常に 0 である。各子機は受信したマスクのうち自分の担当ビットだけを参照して演奏可否を判定する。楽譜のデータそのものは送らず、各子機が自分のパートの譜面配列を保持する分散構成とした。これにより 1 tick あたりの通信は 1 フレームで済み、子機の台数が増えても通信量が変わらない。

表 1 赤外線マスクのビット割り当て

ビット	値	対応する子機
Bit 0	0x01	子機 0 (ピアノ)
Bit 1	0x02	子機 1 (フルート)
Bit 2	0x04	子機 2 (トランペット)
Bit 3	0x08	子機 3 (ドラム)

親機は演奏中、テンポに応じた周期で tick を刻む。tick 間隔 T_{tick} はテンポを BPM として

$$T_{\text{tick}} = \frac{60000}{2BPM} \text{ [ms]} \quad (1)$$

であり、1 tick が 8 分音符 1 個に相当する。120 BPM のとき T_{tick} は 250 ms になる。親機は参加中の子機が 1 台以上ある tick でのみ赤外線フレームを送信する。

2.4 輪唱の実現と参加・退場の状態管理

輪唱は、同じ旋律を時間差で重ねることで成立する。本システムでは、親機の再生開始後に演奏者が子機ごとのボタンを押すことで、その楽器を任意のタイミングで演奏に参加させる方式を採用した。人間の入力タイミングにはばらつきがあるため、参加の反映は次の 8 tick 境界、すなわち拍のまとまりの先頭まで待ってから行う。こうすることで、どのタイミングでボタンを押しても各声部の入りが拍に揃い、輪唱としてかみ合う。退場の反映は、その子機が参加した時点を起点として楽譜 1 周分である 32 tick の倍数の境界に揃える。旋律を途中で切らず、1 周弾き切ってから抜けるための設計である。

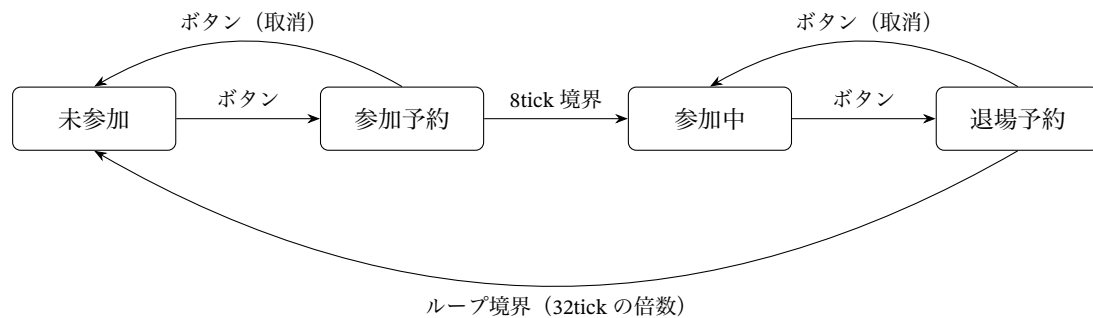


図3 子機1台分の参加状態の遷移

親機が管理する子機1台分の状態遷移を図3に示す。各子機は未参加、参加予約、参加中、退場予約の4状態を巡回し、ボタンを押すたびに予約とその取消が交互に切り替わる。予約中にもう一度押せば取り消せるため、押し間違いにも対処できる。親機の各子機ボタンの隣には状態表示用のLEDを設け、参加の意思がある状態で点灯するようにした。どの楽器が参加予約中あるいは参加中かを物理的に確認できるので、演奏デモの操作が分かりやすくなる。

2.5 楽器音の合成

Processing側では、子機から「音高、音長、音量、譜面位置」のカンマ区切りテキストを受信し、楽器ごとの合成処理で音を再生する。ピアノは倍音の加算合成による減衰音、フルートは倍音構造とADSRエンベロープの調整による持続音、トランペットは倍音比[1.0, 0.6, 0.35, 0.2, 0.1, 0.05]と5Hzのビブラートによる金管らしい音、ドラムはキックとシンバルの2種類の打撃音である。ピアノ音源の詳細は報告者の担当分として3.6節で述べる。

2.6 LCD表示

エンターテインメント性の要素として、図4のように、子機に接続したLCDキャラクターディスプレイに、かえるのドット絵アニメーションと演奏に合わせた歌詞のスクロールを表示する。演奏の進行位置は子機が保持しているため、発音と同じタイミングで表示を更新できる。

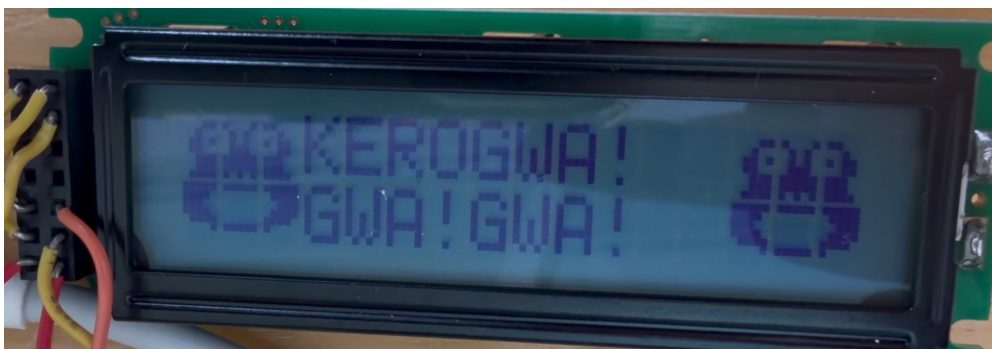


図4 LCDに表示されたかえるのアニメーションと歌詞

3 システム実装

3.1 チーム構成と役割分担

本プロジェクトは6名で実施した。作業はProcessingでの音源作成、子機Arduinoのプログラム、親機Arduinoのプログラム、回路・表示装置の4系統に分け、WBSの管理番号を付けて分担した。役割分担を表2に示す。報告者はピアノ音源の作成とそのArduino通信用関数、ドラムの子機プログラム、およびテンポ管理プログラムを担当した。テンポ管理は渡邊との共同である。第9週末までのチーム全体の作業時間は129時間であり、報告者の累計は35時間であった。

表2 チームの役割分担

担当者	WBS 番号	主な担当
櫛濱 和輝 (報告者)	110, 120, 240, 320	ピアノ音源・通信用関数, ドラム子機, テンポ管理 (共同)
家田 祐希	130, 140, 220, 310	フルート音源・通信用関数・子機, 赤外線管理 (共同)
木下 遼介	150, 160, 230, 410	トランペット音源・通信用関数・子機, 回路 (共同)
久家 力孔	170, 171, 180	ドラム音源 (キック・シンバル), ドラム通信用関数
渡邊 乃斗	250, 310, 320	楽譜進行, 赤外線管理 (共同), テンポ管理 (共同)
手代木 巧	410, 420	回路作成, LED マトリクス・LCD 表示
全員	510, 520	輪唱機能の結合, 結合テスト

開発は、第5週までを計画段階、第6週から第9週までを実装段階とし、第10週に統合と結合テストを行う計画で進めた。WBS第3層のタスク単位のスケジュールを図5に示す。タスク番号は表2のWBS番号に対応する。

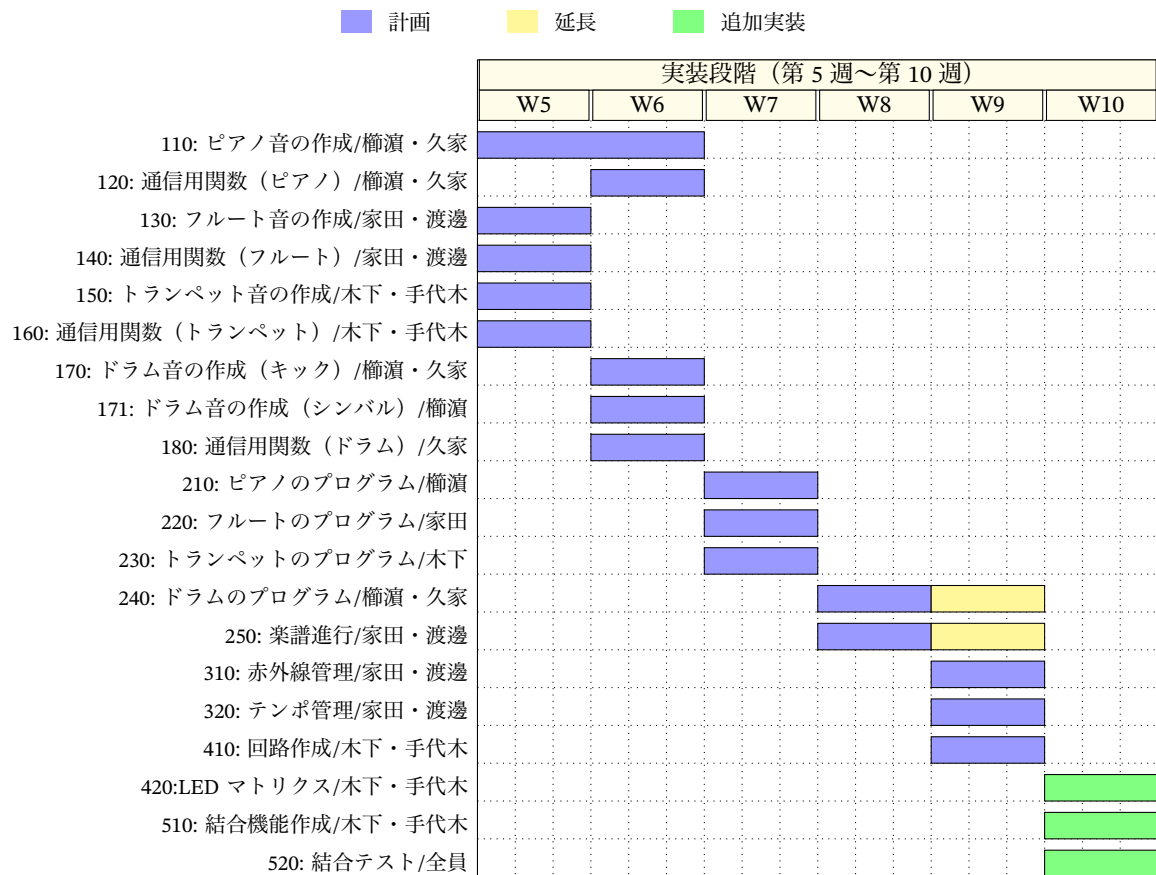


図 5 開発スケジュール

図 5 のバーは計画時の週割当てである。110: ピアノ音の作成は第 5 週の仮実装と第 7 週の音色改善の 2 期間に分けて実施した。実施ではいくつかのタスクが計画より後ろへ延びており、420: LED マトリクス作成は表示不具合の解消に時間を要して第 10 週まで作業が続いた。250: 楽譜進行、310: 赤外線管理、320: テンポ管理の 3 タスクも、第 8 週の結合テストで発覚した課題に対応するため第 9 週まで追加実装を行った。音源作成と通信用関数は計画通り第 7 週末までに完了している。

3.2 親機 Arduino の処理

親機は、演奏の開始とリセット、子機の参加と退場の予約、およびテンポの変更を受け付けながら、全子機に共通の時間基準となる tick を刻み、赤外線フレームを送信する指揮装置である。主要な関数を表 3 に、loop() を 1 周する処理の流れを図 6 に示す。loop() では、シリアルコマンドと各ボタンの入力、および可変抵抗によるテンポ変更を每周監視したうえで、再生中であれば前回の tick からの経過時間を調べ、式 (1) の tick 間隔 T_{tick} に達したときだけ tick() を実行する。参加ボタンとそれに対応するシリアルコマンドは toggleChild に集約されており、図 3 の状態遷移に従って予約とその取消が切り替わる。

tick() では、まず参加予約と退場予約が発火する tick に到達したかを子機ごとに判定して参加状

態を更新し、参加中の子機集合を表すマスクを組み立てる。参加中の子機が1台でもあるときは、遅延計測用の同期パルスを出したのち、tick カウンタの下位 8 ビットとマスクを赤外線 の 1 フレームとして送信する。loop() の末尾では子機ボタン横の LED を参加状態と予約の有無に同期させ、予約の取消を含むどの操作でも表示が確実に切り替わるようにしている。

表 3 親機 Arduino の主要関数

関数名	役割	概要
loop	メインループ	入力の監視と tick 周期の管理を行い、子機 LED の表示を内部状態に同期させる
handleSerialCommand	シリアル入力の処理	1 文字コマンドを解釈し、参加トグルや再生開始、リセットの各関数に振り分ける
handleOrderButtons	参加ボタンの処理	子機ごとのボタンの立ち下がりを検出して toggleChild を呼び出す
toggleChild	参加と退場の予約	子機の状態に応じて参加予約、退場予約、およびそれらの取消を切り替える
handleTempoPot	テンポの調整	可変抵抗の値を平滑化して読み取り、60～150 BPM の範囲でテンポを更新する
doStart	再生の開始	全子機を未参加に初期化し、tick カウンタを 0 に戻して計時を開始する
doReset	リセット	再生を停止し、全子機の参加状態とすべての予約を初期化する
tick	同期信号の送信	予約の発火判定と参加マスクの生成を行い、赤外線フレームを送信する

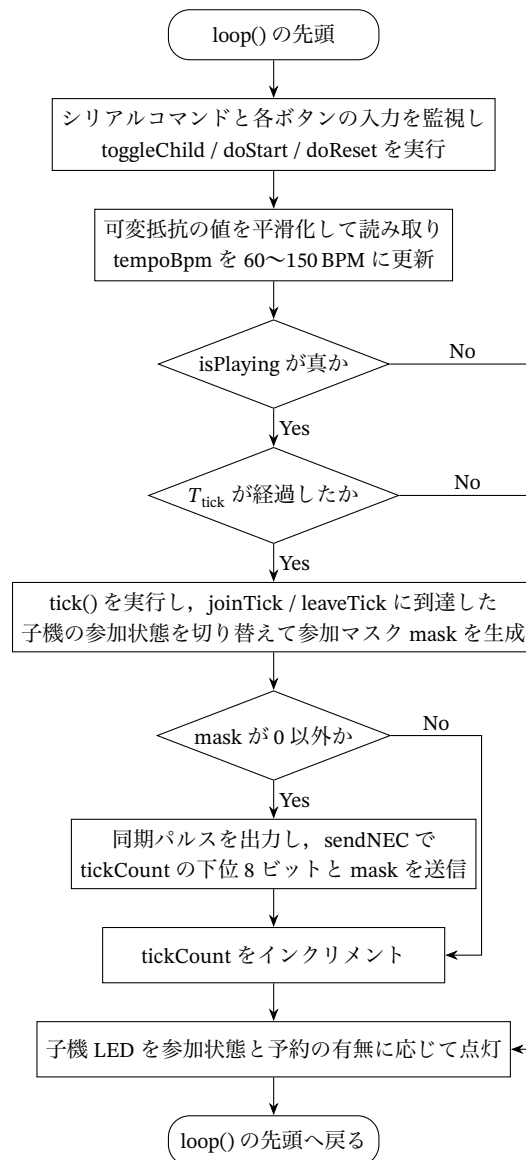


図 6 親機 Arduino の処理フロー

3.3 子機 Arduino の処理

子機 Arduino は、親機からの赤外線フレームを受けて自分のパートを演奏する側の装置である。楽譜のデータは通信では受け取らず、各子機が自分のパートの譜面配列をプログラム内に保持している。受信したマスクのうち自分の担当ビットが立っている tick だけを自分の出番と解釈し、出番の回数を数えるカウンタ `localPos` を 1 つ進めたうえで、譜面配列のその位置の音符情報をシリアル通信で Processing へ送信する。親機の進行にはループの概念がないため、メロディ機は譜面の末尾に達したら `localPos` を 0 へ戻し、子機の側で先頭からの繰り返しを作る。ドラム機は譜面を持たず、出番のたびにキックの合図を送るだけである。子機のプログラムは 4 台で共通であり、冒頭

の機体番号 `childId` を書き換えて各機体へ書き込むことで、メロディ機 3 台とドラム機 1 台を作り分けている。

譜面位置 24 以降は、1tick につき 2 音を発音する倍速区間である。旋律の終盤にあたるこの区間では、1 音あたりの音長を通常の 0.40s から 0.20s へ半分にし、2 音目を直近の受信間隔の半分だけ遅らせて送信することで、16 分音符相当の刻みを作る。待ち時間の基準となる受信間隔は受信のたびに計測するが、演奏の停止と再開のあいだに生じる大きな間隔をそのまま使うと待ち時間が膨らむため、極端な値は捨てて更新する。また、この待ち時間の間にも次のフレームは届きうるので、受信データは復号した直後に退避して受信器をすぐに再開し、待機中も受信を続けて tick の取りこぼしを防いでいる。このほか、親機の同期パルスの立ち上がり時刻を割り込みで記録し、赤外線を受信までの遅延をログへ出力する計測の仕組みも子機側に持たせている。子機の主要な関数を表 4 に、loop の処理の流れを図 7 に示す。

表 4 子機 Arduino の主要関数

関数名	役割	概要
<code>setup</code>	初期化	シリアル通信と赤外線受信を開始し、同期パルス入力割り込みを登録する
<code>loop</code>	受信監視と発音	赤外線フレームを常時監視し、自分の出番の tick で譜面位置を進めて音符情報を送信する
<code>sendNoteToHost</code>	音符情報の送信	音高、音長、音量、譜面位置をカンマ区切りの 1 行にまとめて Processing へ送る
<code>onSyncRise</code>	同期パルスの記録	親機の同期パルスの立ち上がり時刻を記録する割り込み処理で、遅延計測の基準になる
<code>printReady</code>	起動通知	起動時に自分の機体番号とメロディ機かドラム機かの種別をシリアルへ出力する

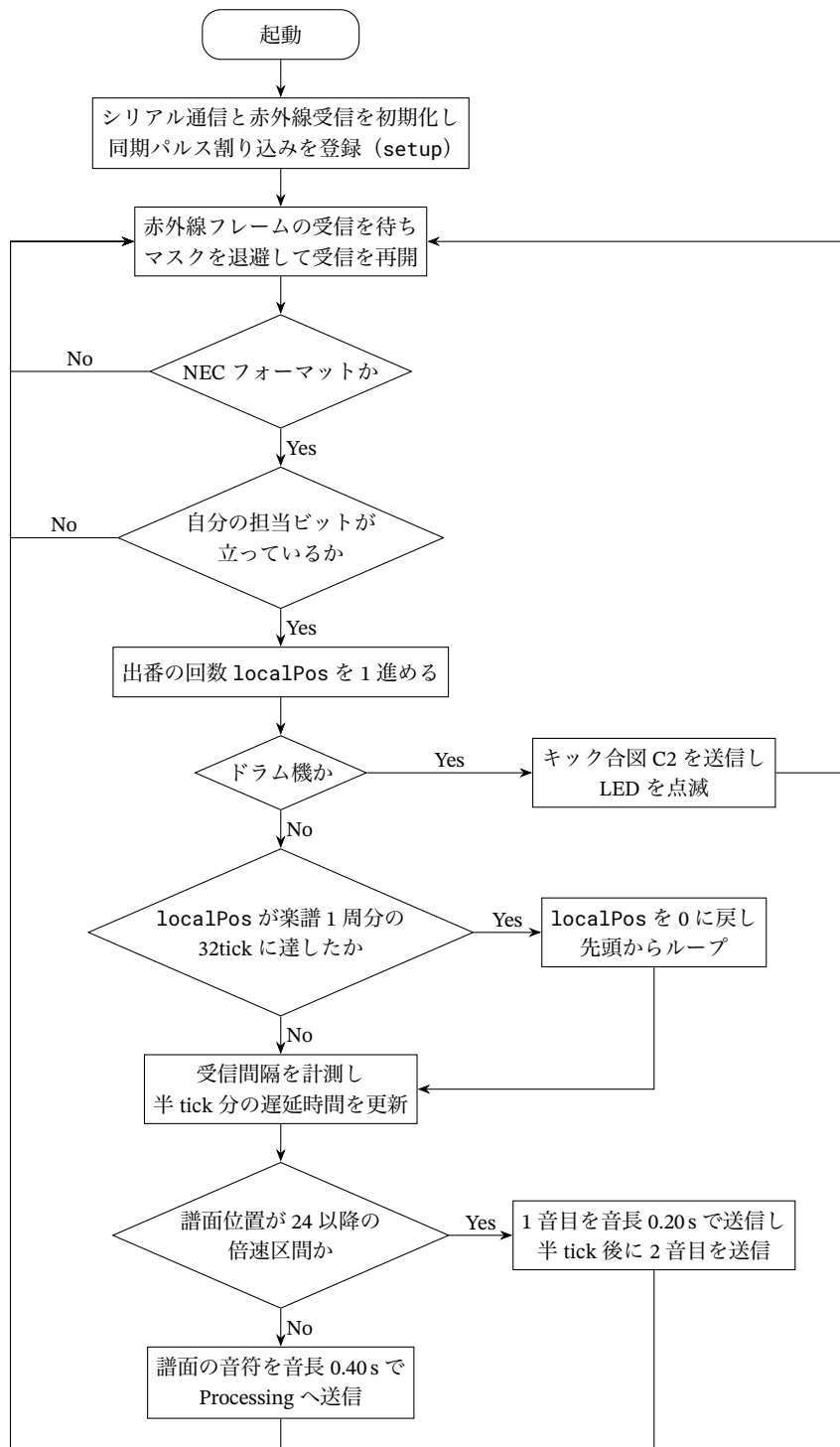


図 7 子機 Arduino の処理フロー

3.4 LCD 表示の処理

表示用の子機は、16 文字 2 行の LCD キャラクタディスプレイ 2 枚を 1 台の Arduino で駆動し、1 枚目にかえるの顔を、2 枚目に歌詞を表示する。顔は HD44780 のカスタム文字機能で作成した 5×8 ドットのパターンを 3 列 2 行に並べて構成し、口の開閉は下段 3 文字を開いた絵柄と閉じた絵柄とで差し替えて表現する。歌詞は楽譜の 32tick と 1 対 1 に対応する文字列テーブルとして保持し、上段に現在の歌詞を、下段に次の歌詞を表示して先読みできるようにした。

表示の進行は親機の赤外線 tick に同期させる。受信したマスクの担当ビットが立った tick でのみ歌詞を 1 つ進めて口パクを行う。歌詞位置 localPos は受信フレームに載った tick 番号の下位 8 ビット tickLow から復元するため、受信を取りこぼしても次の受信で正しい位置へ自己修復する。描画では、HD44780 の CGRAM を書き換えると表示中の文字の形も即座に変わる性質を利用し、顔の位置が変わらない tick では画面を消去せずに口のパターン 3 文字だけを書き換えることで、全再描画によるチラつきを避けている。口パク 2 回ごとに顔を 1 マス右へ移動させ、このときだけ全体を描き直す。顔が右端に達したら左端へ戻して繰り返す。主要な関数を表 5 に、処理の流れを図 8 に示す。

表 5 LCD 表示の主要関数

関数名	役割	概要
setup	初期化	LCD2 枚を初期化してかえるのカスタム文字を登録し、待機画面を表示する
loop	受信と振り分け	赤外線フレームを受信し、担当 tick の判定と各表示処理の呼び出しを行う
showLyric	歌詞表示	LCD2 の上段に現在の歌詞を、下段に次の歌詞を書き込む
drawFrogFull	顔の全再描画	口の開閉に応じたカスタム文字を登録し直し、画面を消去して指定位置に顔を描き直す
setMouth	口パクの更新	CGRAM の口パターン 3 文字だけを書き換え、再描画なしで口を開閉する

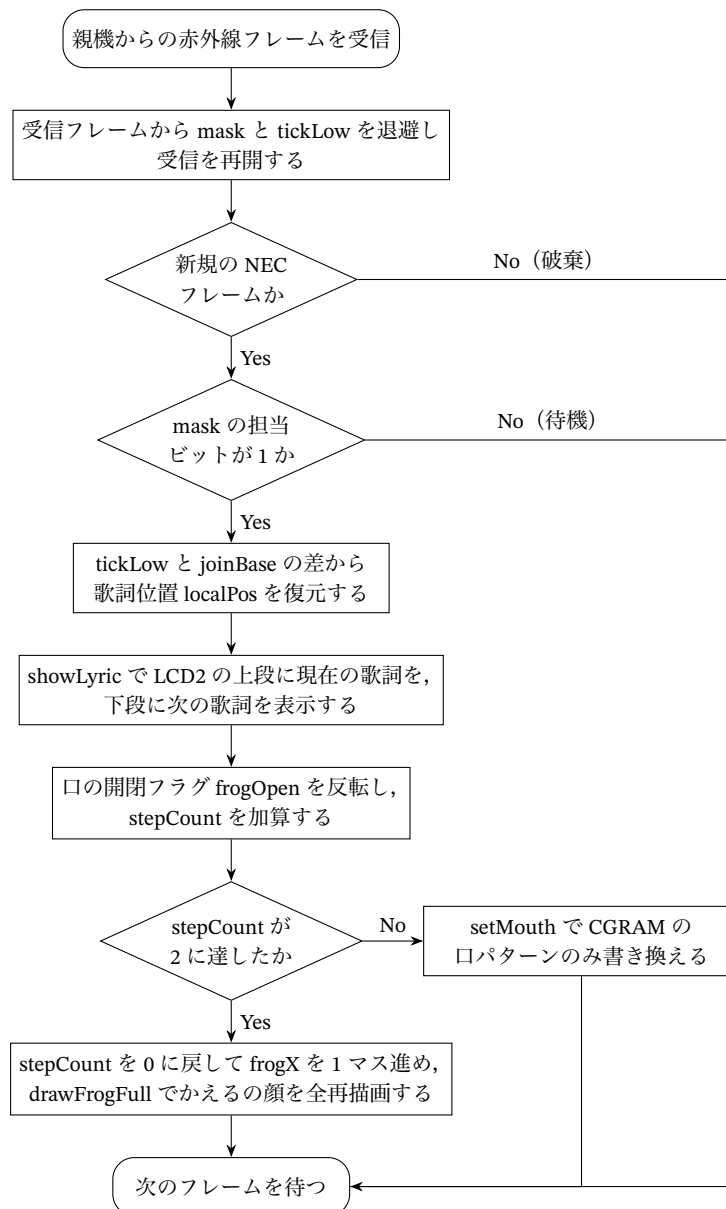


図 8 LCD 表示の処理フロー

3.5 報告者の担当範囲と技術的課題

報告者の担当のうち、ピアノはこのシステムの旋律を担う中心の楽器である。また、テンポ管理と演奏進行の時間設計はすべての楽器に影響する。実装を進めるなかで、解決すべき課題は次の4点に整理された。

- 打鍵後に減衰していくピアノらしい音色を、音源データを使わずに合成で作ること
- 発音時の処理遅延を抑え、演奏のタイミングを崩さないこと

- ・親機と子機の進行が食い違い、特定の音が周期的に抜けるバグを解消すること
- ・休符が休符として聞こえるように、音の余韻を制御すること

以降の節で、それぞれの課題への取り組みを順に述べる。なお、開発支援ツールとして生成 AI を利用した。ピアノ音色の合成アルゴリズムの検討や、バグ修正時の原因候補の洗い出しに用いたが、生成されたコードはそのまま採用せず、実機での聴感確認とシリアルログによる動作確認で検証したうえで、音色を決めるパラメータ類は試聴を繰り返して手動で調整した。検証の具体的な内容は該当する節と 4 章で述べる。このほか、Arduino 公式ドキュメント [7]、IRremote ライブラリのドキュメント [5]、受光モジュールのデータシート [8] を情報源として参照した。

3.6 ピアノ音源の実装

ピアノの音は、多数の倍音が異なる速さで減衰していく音である。正弦波 1 本やオルガン風の持続音では鍵盤楽器に聞こえないため、物理的な性質を模した加算合成を実装した。ピアノ弦の倍音周波数は弦の剛性により整数倍からわずかに高い方へずれることが知られており [9]、この性質はインハーモニシティと呼ばれる [10]。 h 次倍音の周波数 f_h を、基本周波数 f_s とインハーモニシティ係数 B を用いて

$$f_h = hf_s \sqrt{1 + Bh^2} \quad (2)$$

で計算する。 B は音域に応じて変え、中央ハ付近で 2×10^{-4} 程度とした。倍音は 10 次まで重ね、 h 次倍音の振幅は $1/h^p$ に比例させて高次ほど弱くする。指数 p は低音域で 0.9、高音域で 1.4 程度になるように基本周波数から連続的に決めた。

減衰のエンベロープは、打鍵直後の速い減衰とその後の緩やかな減衰の 2 段階を重ねた形とし、時刻 t での振幅係数を

$$e(t) = 0.3e^{-t/0.12} + 0.7e^{-t/\tau_h}, \quad \tau_h = \frac{\tau_1}{1 + 0.6(h-1)} \quad (3)$$

とした。 τ_1 は基本音の減衰時定数であり、残響の長さが低音ほど長くなるように音域から決める。式 (3) の τ_h は次数とともに小さくなるので、高次倍音ほど速く消える。これは実際のピアノで打鍵直後は明るく、時間とともに丸い音色に変わっていく振る舞いに対応する。このほか、実機のピアノが 1 音あたり複数の弦を持つことを模して、音域に応じて 1~3 本の弦をセント単位でわずかにずらして重ねた。さらに打鍵の瞬間のハンマーノイズとして、最初の 15 ms だけローパスフィルタを通した雑音を加え、立ち上がり 3 ms のアタックと、音長経過後に時定数 0.15 s で減衰するダンパーリリースを付けている。

もう 1 つの課題は発音遅延である。上記の合成をすべて発音のたびに計算すると、波形生成に時間がかかり演奏に間に合わないおそれがある。そこで、楽譜に登場する 6 種類の音高と 2 種類の音長の組を Processing の起動時にすべて波形として生成し、Minim の Sampler にキャッシュしておく方式にした。演奏中はキャッシュ済み Sampler の trigger() を呼ぶだけなので、発音時の処理は音源データの再生と同じで済む。Sampler は同時 6 声まで発音できる設定とし、前の音の余韻と次の音

が自然に重なるようにした。未登録の音高が届いた場合はその場で生成してキャッシュに加える安全網も設けている。なお、このピアノ音源は当初オルガン風の持続音で仮実装しており、第9週に本節で述べた合成方式へ差し替えた。差し替え時には楽譜で使う全音高について試聴による検証を行い、全ケースで楽器音として問題ないことを確認している。各波形生成部のコードは付録C,D,E,F, に示す。

3.7 演奏進行の不具合修正

結合テストの過程で、演奏の時間管理に関わる不具合を2件修正した。いずれも症状の観察からログで原因を絞り込んだもので、修正自体は小さいが、輪唱の成立には欠かせない修正だった。

1件目は、特定の位置の音が周期的に鳴らない症状である。旋律の途中、譜面位置16や24の音が数周に一度抜けて聞こえた。子機のシリアルログで譜面位置の進み方を追ったところ、親機は32tickを楽譜1周として進行しているのに対し、子機は1周を33tickと解釈していることが分かった。子機側でループ末尾の休みを表す定数を2tickとしていたため、旋律31tickとの合計が33tickになっていたのが原因である。周回のたびに子機の位置が親機に対して1tickずつ後ろへずれ、ずれの蓄積がちょうど一周する周期で特定位置の音が落ちる、という症状につながっていた。この定数を1に修正して両者を32tickに統一し、症状は解消した。第9週の4子機実機テストでも再発していない。

2件目は、休符が休符として聞こえない問題である。本システムの音長の既定値は0.40sであり、120BPMでのtick間隔0.25sより長い。前の音の余韻が次のtickまで残るため、譜面上の休符Rのタイミングでも直前の音が鳴り続け、旋律の切れ目が埋まってしまっていた。そこでProcessing側で休符Rを受信した時点で、発音中の音を即時に停止する処理を入れた。オルガン風音源の時点ではnoteOff()の即時呼び出しとして実装し、ピアノ音源への差し替え後もSamplerのstop()で同じ動作を維持している。音が鳴っている最中に切る処理なので不自然に聞こえる心配があったが、ピアノのダンパーで音を止めた場合と似た切れ方になり、聴感上の違和感はなかった。

3.8 テンポ管理

テンポ管理は、親機のtick周期を演奏中に変更する機能である。当初はボタンの押下でテンポを段階的に増減する方式を実装していたが、デモで聴かせる際にテンポが跳びとびに変わるのは不自然だったため、第9週に可変抵抗による連続調整へ変更した。10kΩのシングルターン可変抵抗をアナログ入力に接続し、読み値 v (0~1023) をテンポの範囲60~150BPMへ線形に対応付ける。

$$BPM(v) = 60 + \left\lfloor \frac{(150 - 60)v}{1023} \right\rfloor \quad (4)$$

読み取りは50ms間隔とし、読み値のノイズによるテンポの揺れを抑えるため、

$$s_n = \frac{3s_{n-1} + v_n}{4} \quad (5)$$

の指数移動平均で平滑化してから式(4)に与える。親機のtick周期の計算は毎ループ現在のテンポ変数を参照しているため、つまみの操作は次のtickから即座に反映される。シリアルモニタへのロ

グ出力は 2BPM 以上変化したときに限定し、微小な揺らぎでログが埋まらないようにした。

なお、親機の進行処理は `delay()` で待つのではなく、`millis()` で前回 tick からの経過時間を監視する方式である。待ち時間の間もボタン入力と可変抵抗の読み取りを続ける必要があるため、テンポ変更・参加操作・tick 送信が互いをブロックしない構造になっている。

3.9 輪唱開始方式の試作と設計変更

輪唱の開始方法については、開発の途中で設計を大きく変えた経緯がある。当初の計画では、演奏開始前にボタンで各楽器の順番を登録し、登録順に固定の時間差で追いかける方を想定していた。報告者はこの系統の親機プログラムに対して、押下順に応じて各子機の出だしのずれを動的に割り当てる実装を試作した。子機固有の固定遅延値として持っていた配列を、押された順番 N に対する割当 $\{0, 8, 16, 20\}$ tick へ読み替えることで、どのボタンを最初に押してもその楽器が先頭声部として位置 0 から歌い出す、本来のカノンの形を実現するものである。

一方で、チーム内の検討により、輪唱の定義は「固定順で参加する」ものではなく「任意の順・任意のタイミングで参加できる」べきだという結論になり、第 7 週末にアーキテクチャを再生開始後の随時参加ボタン方式へ変更した。この方式では子機は演奏していない間も常に曲位置を共有しているため、途中から参加しても、その時点の位置から自然に演奏へ加われる。押下順カノン方式の試作は採用に至らなかったが、2 案を実機で比較したことで、随時参加方式の方が演奏デモとしての自由度が高いことを確認したうえで移行できた。最終版の親機プログラムはこの随時参加方式であり、報告者が試作で得た 8tick 境界への参加アライメントの考え方は、そのまま参加予約の量子化として引き継がれている。

3.10 担当部分以外の実装と工夫

システムの残りの部分について、チームの実装を概観する。赤外線管理プログラムは渡邊と家田が担当し、親機からマスクと tick 番号を一齐送信する方式や、参加・退場を OFF・QUEUED・ACTIVE の 3 状態で管理する仕組みを実装した。楽譜進行プログラムは渡邊が担当し、スタート信号を受けた tick の記録と、現在の tick との比較で出力音を決める機能を追加した。

フルートは家田の担当で、倍音構造と ADSR の調整により息の混じった柔らかい音色を作り、アタックタイムを短くしてテンポへの追従性を改善した。トランペットは木下の担当で、倍音比とビブラートによる音色合成に加え、音の切り替え時に発生していたプチという雑音を、Minim の出力ラインに 0.03 s のフェードアウトを入れることで解消した。また、トランペット系では Arduino から 29 音の楽曲データを一括送信するため、シリアル通信速度を 921600 bps とし受信配列への格納を確認している。ドラム音源は久家の担当で、キックとシンバルの 2 種を合成し、Arduino から受信した整数値で再生し分ける Processing を実装した。回路と表示装置は手代木の担当で、親機・子機の回路作成と、LCD 表示プログラムの作成を進めた。

回路面では、第 6 週に赤外線受光モジュールの故障が 3 台見つかり、新品への交換と動作確認を家田が行った。こうした部品不良の切り分けができたのは、受光モジュール単体の受信テストを単体テストとして分離していたためである。

4 評価方法と結果

4.1 評価方法

本システムの評価は、計画段階で定義した有効性指標 (MOE)、性能指標 (MOP)、技術性能指標 (TPM) に基づいて行う。有効性指標 MOE を表 6 に示す。

表 6 有効性指標 (MOE) の定義

MOE	有効性指標
MOE1	違和感なく聞くことができる
MOE2	可変テンポに対応している
MOE3	楽器音らしい音色である
MOE4	歌詞・アニメーションが演奏に対応している

各 MOE に対応する性能指標 MOP を表 7 に定めた。MOE1 には MOP1 (演奏タイミング誤差) と MOP2 (同期信号の受信成功率) が、MOE2 には MOP3 (テンポ変更への追従時間) が、MOE3 には MOP4 (音色の再現性) が、MOE4 には MOP5 (歌詞・アニメーション同期遅延) がそれぞれ対応する。演奏タイミング誤差の目標値 30 ms は、合奏で聴衆がずれを知覚し始める閾値が 20～30 ms 程度とされていること [1] を根拠とし、映像と音の同期に関する研究 [28] でも同様の知見が示されている。LCD 表示の同期許容範囲 55 ms は、楽器演奏における音と映像のずれの知覚に関する報告 [11] と、アンサンブル演奏における音の遅延条件の研究 [29] に基づく。

表 7 性能指標 MOP と目標値

MOP	性能指標	目標値	最終状態
MOP1	演奏タイミング誤差	30 ms 以内	平均 28 ms (10000 回計測, 達成)
MOP2	同期信号の受信成功率	95 % 以上	100 % (10000 回中 10000 回成功, 達成)
MOP3	テンポ変更への追従時間	1 拍以内	1 拍以内に追従 (達成)
MOP4	音色の再現性	聴衆の過半数が楽器と判断	全 5 音色で過半数が 4 以上評価 (達成)
MOP5	歌詞・アニメーション同期遅延	55 ms 以内	約 50.1 ms (達成)

検証は単体テスト、結合テスト、システムテストの 3 段階で計画した。単体テストでは各音源と通信機能を個別に確認し、結合テストでは親機・子機間の同期と子機・Processing 間の接続を確認する。システムテストでは全構成を接続して課題曲の通し演奏を行う。

4.2 単体テストの結果

第 5 週から第 8 週にかけて実施した単体テストの結果を表 8 に示す。音源系では、フルートの音色聴感テスト、トランペットの音色合成テストと切り替えノイズテスト、ドラムの単体再生テストがそれぞれ完了した。トランペットの切り替えノイズは、フェードアウト処理の追加によってテスト上で雑音を確認されなくなり、MOP5 に関連する問題点として管理していたものが解消された。通信系では、921600 bps でのシリアル通信テストで 29 音の楽曲データが欠落なく受信・格納され

ることを確認した。また、赤外線受光モジュールの故障3台の交換後に受信テストを再実施し、回路の正常動作を確認している。ピアノ音源は差し替えの際に、楽譜で使う全音高と2種類の音長の組について試聴で確認した。

表8 テストの実施結果

実施日	テスト内容	結果	備考
5/20～5/25	フルート音色聴感テスト	成功	倍音構造・ADSR調整完了
5/26	トランペット音色合成テスト	成功	倍音比とビブラートの確定
5/26	音切り替えノイズテスト	成功	0.03s フェードアウトで雑音解消
5/26	シリアル通信テスト (921600 bps)	成功	29音を欠落なく受信
6月初旬	ドラム音色単体テスト	成功	キック・シンバルの再生確認
6/17	受光モジュール交換後の通信テスト	成功	故障3台を交換
6/19	3子機の輪唱テスト	部分成功	ドラムを除く3楽器で動作
6/23	4子機の輪唱実機テスト	成功	音抜け・休符抜けの再発なし

4.3 結合テストと実機演奏の結果

結合テストは2段階で行った。第7週末の3子機テストでは、ピアノ・フルート・トランペットの3楽器での輪唱開始を実機で確認した。この時点ではドラムが未結合であり、部分成功という扱いである。続く第9週の6月23日に、親機1台と子機4台の全構成による輪唱の実機テストを実施した。ボタンの押下順にピアノ・フルート・トランペット・ドラムが順次参加するカノン演奏を通して行い、聴感上の破綻なく演奏できることを全員で確認した。3.7節で述べた位置抜けと休符抜けは、このテストで再発していない。参加・退場のトグル操作についても、予約とその取消、途中退場後の再参加が状態表示LEDの点灯と一致して動作した。

テンポ管理については、可変抵抗のつまみ操作で演奏中のテンポが60～150BPMの範囲で連続的に変わることを確認した。式(1)のtick周期は毎ループ現在のテンポを参照するため、追従の遅れはつまみ読み取りの平滑化に伴う数百ms程度であり、1拍以内というMOP3の目標を満たす動作である。ただしこの追従時間もログからの定量値としては算出しておらず、シリアルモニタ上の値の変化と聴感での確認にとどまっている。

4.4 定量評価の結果

7月13日に実施した定量計測の結果を以下に示す。赤外線通信誤差は親機の送信パルスと子機の受信復号完了の時刻差をマイクロ秒単位でログ出力し、10000回の統計から算出した。LCD同期誤差は、演奏中のシステム全体を上方から動画撮影し、発音タイミングとLCD表示のフレーム差分から計測した。

4.4.1 同期・通信の評価結果

赤外線通信による同期の定量評価結果を表9に示す。赤外線通信誤差は10000回の計測の結果、平均28msとなり目標の30ms以内を達成した。Sony SIRC 12ビットの採用によりフレーム長を最

小化したことが低遅延に寄与している。信号受信率は 10000 回中 10000 回の成功で 100 %を記録し、テンポ変更への追従も 1 拍以内に収まった。

表 9 同期・通信の評価結果

評価項目	目標	結果	判定
赤外線通信誤差	平均 30 ms 以内	平均 28 ms (10000 回計測)	達成
信号受信率	95 %以上	100 % (10000 回中 10000 回成功)	達成
テンポ変更追従	1 拍以内	1 拍以内に追従	達成

4.4.2 音色の評価結果

楽器音の音色について、18 人を対象としたアンケート評価（5 段階）の結果を表 10 に示す。全楽器で「4 以上の割合が過半数」という目標を達成した。

表 10 音色の評価結果（18 人アンケート，5 段階評価）

楽器音	評価平均	4 以上の割合	判定
ピアノ	3.78	61.1 % (11/18 人)	達成
フルート	4.33	83.3 % (15/18 人)	達成
トランペット	3.61	61.1 % (11/18 人)	達成
キック	3.72	61.1 % (11/18 人)	達成
シンバル	4.00	66.7 % (12/18 人)	達成

4.4.3 LCD 同期の評価結果

LCD 表示と演奏の同期に関する評価結果を表 11 に示す。同期誤差は目標 55 ms 以内に対して約 50.1 ms であり、十分な精度で同期できている。

表 11 LCD 同期の評価結果

評価項目	目標	結果	判定
LCD 同期誤差	約 55 ms 以内	約 50.1 ms	達成

4.4.4 全体評価

表 12 に評価分類ごとの総合判定を示す。同期・通信制御では赤外線通信誤差平均 28 ms（目標 30 ms 以内）、受信率 100 %（目標 95 %以上）と全項目で目標を達成した。LCD 表示同期も約 50.1 ms と目標 55 ms 以内を達成した。音色表現は過半数の評価者から良好と判定されたが、トランペットとキックの評価平均が 3.6 台にとどまっており、改善の余地がある。

表 12 全体評価

評価分類	結果	判断
同期・通信制御	演奏誤差 28 ms, 受信率 100 %, テンポ追従 1 拍以内	達成
音声出力・音色	過半数 4 以上は全音色で達成	概ね達成
LCD 表示同期	同期誤差 約 50.1 ms	達成
総合評価	主要目標達成, 音色表現に改善余地	概ね達成

4.5 定量指標の計測状況

MOP1（演奏タイミング誤差）と MOP2（信号受信率）の定量計測は 7 月 13 日に実施した。親機には赤外線送信の直前に同期パルスを出す端子を設け、子機側では割り込みでパルスの立ち上がり時刻を記録し、赤外線フレームの受信復号が完了した時刻との差をマイクロ秒単位でログ出力する機構を用いた。10000 回の送受信ログから演奏タイミング誤差の統計を取り平均 28 ms を得た。あわせて送信回数に対する受信回数の比から受信成功率 100 % を算出した。技術性能指標 TPM として監視してきた項目のうち、シリアル通信速度は目標通り動作確認済み、親機と子機の tick 数の整合は 3.7 節の修正で達成済みである。

5 考察

5.1 マスク方式による同期の有効性

本システムの同期方式は、楽譜データを通信せず、参加楽器のマスクだけを毎 tick 送るものである。4 子機の実機テストで輪唱が破綻なく成立したことから、この情報量の絞り込みは有効だったと考える。Sony SIRC 12 ビットの 1 フレームに必要な情報がすべて収まるため、子機の台数を増やしてもフレーム長も送信頻度も変わらない。コマンド部 7 ビットのうち下位 4 ビットをマスクに使用しているので、上位 3 ビットも活用すれば理論上は 7 台までこの構成のまま拡張できる。楽譜を子機に分散させたことは、通信量の削減だけでなく、子機単体でのテストを可能にするという開発上の利点もあった。各楽器の担当者が自分の子機と Processing だけで音源の開発を進められたのは、この構成によるところが大きい。組込みシステム教育において実践的な開発経験の重要性が指摘されており [20]、本システムのような分散型の構成は各自が独立してテスト・開発を進められる点で教育的にも有効である。

一方で、この方式は子機が受信回数を数えて自分の位置を進める作りに依存している。赤外線の受信に 1 回失敗すると子機の位置が 1 tick 遅れ、以後もずれたまま演奏が続く。現状の Sony SIRC 12 ビット構成ではアドレス部 5 ビットを常に 0 としているが、このアドレス部に tick 番号の下位ビットを載せて送り、子機側で受信のたびに tick 番号から自分の譜面位置を計算し直すようにすれば、受信失敗が起きても次のフレームで自己修復できる。今後の改善点の中では優先度が高い。

5.2 tick 数不整合の教訓

3.7 節の位置抜けバグは、親機と子機が「楽譜 1 周分の tick 数」という同じ意味の値を、別々のファイルに別々の定数として持っていたことが根本原因である。修正は定数 1 つの変更で済んだが、症状は数周に一度しか現れないため、原因の特定には子機のログから位置の進み方の周期性を読み取る必要があった。親機 32 に対して子機 33 という 1tick の差は、単体テストでは決して現れず、実機で長く演奏して初めて聞こえてくる。設計書に定数の対応関係を明記して両側から参照すること、そして結合した状態で数周分を通して聴くテストを早い段階に置くことが、同種の不整合への対策になると考える。

5.3 音長の固定値とテンポ可変の関係

休符抜けの問題は、音長の既定値 0.40 s が tick 間隔 0.25 s より長いという時間設計の矛盾から生じた。休符受信時の即時停止で聴感上は解消したが、これは対症的な処置でもある。本システムはテンポを 60~150 BPM まで連続可変としたので、tick 間隔は式 (1) より 500 ms から 200 ms まで変わる。音長を固定値のままにすると、遅いテンポでは音符間に隙間が生まれ、速いテンポでは余韻の重なりが増える。音長を tick 間隔に比例させて、テンポに追従した長さで発音する方式が本質的な改善になる。子機は直近の受信間隔を計測して倍速区間の遅延に使っており、この値を音長の算出にも流用できるはずである。

5.4 ピアノ音源の方式選択

ピアノ音源で採った事前レンダリング方式は、音色の複雑さと発音遅延の両立という点でうまく働いた。発音時の処理が Sampler の trigger() だけになるため、加算合成の計算量が演奏タイミングへ影響しない。一方でこの方式には、音高・音長・音量の組ごとに波形を持つ必要があるという制約がある。今回の楽曲は 6 音高 × 2 音長で済んだが、曲目を増やすならキャッシュの生成時間とメモリが増えていく。また、打鍵の強弱で音色が変わるベロシティ表現は現状持っていない。強弱を数段階の波形として持つか、再生時の音量係数で近似するかは、音質と資源のトレードオフとして今後検討したい。

5.5 定量評価から得られた知見

7 月 13 日の定量計測により、いくつかの新たな知見が得られた。

赤外線通信方式を Sony SIRC 12 ビットに変更したことで、演奏タイミング誤差の目標値 30 ms を平均 28 ms で達成できた。NEC フォーマット（固定 32 ビット）に対し、SIRC は 12 ビットとフレーム長が短く、また NEC が持つ反転バイトによるエラー検出を省いているため、送受信にかかる時間が短縮された。信号受信率は 10000 回の試行で全て成功し 100 % を記録した。送受信距離が適切に保たれていれば通信エラーは発生せず、室内に人数がいても、送受信経路上に人体や機材などの部分的な遮蔽物がある状態でも正確に受信できることが確認された。演奏側が毎拍ごとに同期信号を受信する設計により、テンポ変更にも素早く追従する動作が確認された。

音色評価では、倍音構造や ADSR エンベロープの調整により、全 5 種の楽器音が過半数の評価者から自然な音色として認識された。特にフルートは評価平均 4.33 と高い評価を得た。一方で、トランペット（平均 3.61）とキック（平均 3.72）はやや低い結果となり、倍音構造のさらなる調整が必要であることが分かった。

LCD 表示の同期誤差は約 50.1 ms と目標を十分に下回ったが、演奏中の処理負荷が増大した場合に LCD 表示遅延が拡大する可能性については、今後の検証課題として残されている。

5.6 システムの限界と改善案

最後に、本システムに残された課題を整理する。評価面では、MOP1（演奏タイミング誤差）と MOP2（信号受信率）の定量計測を 7 月 13 日に実施し、いずれも目標を達成した。ただし、Sony SIRC には NEC のようなエラー検出がないため、屋外など赤外線ノイズの多い環境での信頼性は未検証である。機能面では、前述のアドレス部 5 ビットを活用した tick 番号による位置の自己修復を実装すれば、受信失敗への耐性が上がる。さらに、楽譜が子機のプログラムに固定配列として埋め込まれている点も、曲目の追加を考えると改善の余地がある。楽譜データを外部から書き換えられる形式にすれば、同じハードウェア構成のまま演奏曲を増やせる。

6 おわりに

本プロジェクトでは、複数の機器をリアルタイムに同期させる仕組みを合奏という題材で実現することを目的として、赤外線によるマスク同期を用いた 5 台構成の Arduino 輪唱演奏システムを制作した。赤外線ブロードキャストによる 1 対多同時送信で Arduino 間の同期と安定した通信を実現し、楽曲データの通信なしで「かえるのうた」の輪唱を成立させた。演奏者はボタンで各楽器を任意のタイミングで参加・退場させることができ、可変抵抗によるテンポの連続調整にも対応した。

報告者は、倍音の加算合成と事前レンダリングによるピアノ音源、ドラム子機のプログラム、可変抵抗によるテンポ管理を実装し、親機と子機の tick 数不整合による周期的な音抜けと、音の余韻による休符抜けの 2 つの不具合を修正した。

7 月 13 日に実施した定量評価により、演奏タイミング誤差平均 28 ms（目標 30 ms 以内）、信号受信率 10000 回中 10000 回成功（100 %）、LCD 同期誤差約 50.1 ms（目標 55 ms 以内）と主要な到達目標を全て達成した。同期・通信・LCD 表示は全項目で目標を達成し、音色表現も過半数の評価者から良好と判定された。Sony SIRC 12 ビットの採用によりフレーム長を最小化しつつ、屋内環境で十分な通信品質を確保でき、赤外線通信による輪唱システムは有効であることが実証された。

今後の課題として、トランペットとキックの音色改善（倍音構造の調整）、演奏中の処理負荷増大時の LCD 表示遅延の検証、tick 番号を用いた譜面位置の自己修復、およびテンポに追従する音長制御が挙げられる。

7 変更履歴

本報告書の主要な変更履歴を表 13 に示す。

表 13 主要な変更履歴

日付	種別	変更内容
5/13	設計	赤外線通信方式の基本設計を策定
5/20	実装	フルート音色の聴感テスト完了
5/26	実装	トランペット音色合成テスト完了
5/26	修正	トランペット音切り替えノイズを解消 (0.03 s フェードアウト)
5/26	実装	シリアル通信テスト (921600 bps) 完了
6 月初旬	実装	ドラム音色 (キック・シンバル) 単体テスト完了
6/17	修正	赤外線受光モジュール故障 3 台を交換・動作確認
6/19	テスト	3 子機での輪唱テスト (部分成功)
6/23	テスト	4 子機の輪唱実機テスト成功
6 月下旬	修正	tick 数不整合バグを修正 (子機 33tick → 32tick)
6 月下旬	修正	休符が聞こえない問題を修正 (即時停止処理追加)
6 月下旬	設計変更	輪唱開始方式を押下順カノンから随時参加方式へ変更
7 月初旬	実装	ピアノ音源をオルガン風から加算合成方式へ差し替え
7 月初旬	実装	テンポ管理をボタン式から可変抵抗式へ変更
7 月初旬	実装	LCD 表示プログラムの統合
7/8	計測	音色アンケート実施 (18 人, 5 段階評価)
7/8	計測	LCD 同期誤差の計測 (約 50.1 ms, 動画フレーム差分法)
7/13	計測	同期・通信の定量評価実施 (誤差平均 28 ms, 受信率 100 %, 10000 回計測)

参考文献

- [1] Rasch, R. A., “Synchronization in performed ensemble music,” *Acustica*, Vol.43, No.2, pp.121–131, 1979.
- [2] 一般社団法人音楽電子事業協会 (AMEI), “MIDI 規格,” <https://amei.or.jp/midistandardcommittee/>, 参照 2026 年 6 月.
- [3] SB-Projects, “IR Remote Control Theory: NEC Protocol,” <https://www.sbprojects.net/knowledge/ir/nec.php>, 参照 2026 年 5 月.
- [4] SB-Projects, “IR Remote Control Theory: Sony SIRC,” <https://www.sbprojects.net/knowledge/ir/sirc.php>, 参照 2026 年 7 月.
- [5] Arduino-IRremote Community, “IRremote Arduino Library,” <https://github.com/Arduino-IRremote/Arduino-IRremote>, 参照 2026 年 5 月.
- [6] Di Fede, D., “Minim: An audio library for Processing,” <http://code.compartmental.net/minim/>, 参照 2026 年 5 月.
- [7] Arduino, “Arduino Documentation,” <https://docs.arduino.cc/>, 参照 2026 年 5 月.
- [8] OptoSupply, “OSRB38C9AA Infrared Receiver Module Datasheet,” 参照 2026 年 6 月.
- [9] Fletcher, H., Blackham, E. D. and Stratton, R., “Quality of Piano Tones,” *The Journal of the Acoustical Society of America*, Vol.34, No.6, pp.749–761, 1962.
- [10] 西口磯春, “ピアノの音響とその物理モデル,” *RIST ニュース*, 神奈川工科大学, No.56, pp.3–15, 2014.
- [11] Arrighi, R., Alais, D. and Burr, D., “Perceptual synchrony of audiovisual streams for natural and artificial motion sequences,” *Journal of Vision*, Vol.6, No.3, pp.260–268, 2006.
- [12] 三井一希, 佐藤和紀, 萩原丈博, 竹内慎一, 堀田龍也, “IoT ブロックを活用したプログラミング教育の試行,” *デジタル教科書研究*, Vol.7, 2018.
- [13] 板垣翔大, 中村好則, 安藤明伸, 門田和雄, “小学校における IoT 教材を活用した PBL 型授業に関する探索的研究,” *日本教育工学会論文誌*, Vol.44, No.3, 2021.
- [14] 宮川洋一, 森山潤, “STEM, STEAM, エンジニアリング教育概念の比較教育からの論点整理,” *日本産業技術教育学会誌*, Vol.62, No.3, p.197, 2020.
- [15] 中村大輝, 松浦拓也, “学校教育における「ものづくり」を中心とした STEAM 教育実践,” *科学教育研究*, Vol.37, No.3, 2022.
- [16] 坂村健, “INIAD : IoT 時代の新しい学び舎,” *情報の科学と技術*, Vol.67, No.11, p.582, 2017.
- [17] 加藤直樹, 松田孝, 上野朝大, 濱田大地, “小学校理科におけるワンボードマイコンを用いるプログラミング教育の提案,” *初等教育カリキュラム研究*, Vol.1, p.37, 2019.
- [18] 西仁司, 清水幹郎, 青山義弘, 小松貴大, 高久有一, 斉藤徹, “マイコンでのロボット制御を通じた情報工学の基礎教育,” *第 31 回ファジィシステムシンポジウム講演論文集*, WC4-2, 2015.
- [19] 紺谷正樹, 熊谷浩二, 山本利一, 山口哲男, “無線通信インタフェースによるプログラム転送を

- 可能にした教材の開発とセキュリティに関する授業実践,” 日本産業技術教育学会誌, Vol.66, No.1, pp.37-47, 2024.
- [20] 小林敬太, 佐藤賢文, 神戸英利, 三井浩康, “組込みシステム技術者育成のための実践的カリキュラム,” 工学教育, Vol.61, No.4, p.36, 2013.
 - [21] 木戸冬子, “大学におけるハッカソン実施の試み：ハッカソン研究会の活動,” 研究 技術 計画, Vol.30, p.138, 2015.
 - [22] 松本慎平, 中島亨輔, Azelin Mohamed Noor, “オンラインコミュニケーションツールを用いた英語によるプログラミングハッカソンの実践,” コンピュータ利用教育学会, Vol.51, p.75, 2022.
 - [23] 野口孝文, 布施泉, 千田和範, 稲守栄, “IoT 機能を持つロボットを用いた協調学習環境,” 教育システム情報学会誌, Vol.37, No.2, pp.106-119, 2020.
 - [24] 日下菜穂子, 末宗佳倫, 下村篤子, 上田信行, “プログラミング教育を介した多世代が教えあう協調学習の生涯発達における可能性,” 発達心理学研究, Vol.31, No.4, p.201, 2020.
 - [25] 井垣宏, 武元貴一, 上田悠貴, “基礎的な IoT 教育のためのチーム開発を重視した PBL 授業の提案,” コンピュータソフトウェア, Vol.35, No.1, p.54, 2018.
 - [26] 吉川祐樹, 川勝望, 林和彦, “全学科全学年に横断した PBL 型授業による学生の主体性の向上について,” 工学教育, Vol.69, No.2, p.2, 2021.
 - [27] 橋本武彦, “データサイエンティストの採用・育成におけるハッカソンの活用,” データサイエンス研究, Vol.3, No.1, p.25, 2019.
 - [28] 桑原圭裕, “映像と音の同期 — 「動画先行の原則」 の模擬と応用,” 映像学, Vol.102, pp.54-74, 2019.
 - [29] 長尾翼 他, “音の遅延条件がアンサンブル演奏に与える影響に関する検討,” 日本音響学会講演論文集, pp.997-998, 2012.

A 親機 Arduino のスケッチ

親機のスケッチを以下に示す。報告者の担当箇所は、可変抵抗によるテンポ管理を行う `handleTempoPot()` と、子機の状態表示 LED の制御である。

```
1 #define DECODE_SONY // Sony SIRCプロトコルのデコードを有効化
2 #define IR_SEND_PIN 3 // 赤外線LEDの送信ピンをD3に設定
3 #include <IRremote.hpp> // 赤外線送受信ライブラリを読み込む
4
5 const int NUM_CHILDREN = 4; // 子機の台数(ピアノ/フルート/トランペット/ドラム)
6
7 const int PIN_BTN_ORDER[NUM_CHILDREN] = {4, 5, 6, 7}; // 子機ごとの「参加/退場」トグルボタン
8 // 各子機ボタン横のLED (D4→A0, D5→A1, D6→A2, D7→A3)。
9 // ボタン押下毎に必ずトグル (参加予約/取消/退場予約/取消 のいずれでも反転)。
10 const int PIN_LED_CHILD[NUM_CHILDREN] = {A0, A1, A2, A3}; // 参加状態表示LEDのピン配列(上コメント参照)
11 const int PIN_BTN_START = 8; // 演奏開始ボタンのピン(D8)
12 const int PIN_POT_TEMPO = A5; // BPM可変抵抗器 (uxcell 10kΩ シングルターン)。回転で 60~150 BPM
13 const int PIN_BTN_RESET = 10; // リセットボタンのピン(D10)
14 const int LED_INDICATOR = 13; // 動作確認用の内蔵LED(D13)
15 const int PIN_SYNC_OUT = 9; // 子機D3への同期パルス出力 (遅延計測用)
16
17 const int QUANTIZE_TICKS = 8; // 参加予約のアライメント(0,8,16,...)
18 // 子機の TOTAL_TICKS (=楽譜1周分のIR tick数) と一致させる必要がある。
19 // 退場予約はこの倍数(子機の譜面ループ末尾)にアライメントする。
20 const int SCORE_TICKS = 32; // 子機の楽譜1周分のtick数(退場アライメントの基準)
21 const int TEMPO_MIN_BPM = 60; // テンポ下限(可変抵抗の最小側)
22 const int TEMPO_MAX_BPM = 150; // テンポ上限(可変抵抗の最大側)
23 const unsigned long POT_READ_INTERVAL_MS = 50; // 可変抵抗の読み取り間隔(50ms)
24 // Sony SIRC 12bit: address(5bit) + command(7bit)。mask は command に載せる
25
26 bool isActive[NUM_CHILDREN] = {false, false, false, false}; // 参加済みフラグ
27 long joinTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 参加予定tick (未予約は-1)
28 long leaveTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 退場予定tick (未予約は-1)
29 long joinedAtTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 実際に参加発火したtick(退場アライメント計算用)
30
31 bool isPlaying = false; // 演奏中かどうかのフラグ
32 int tempoBpm = 120; // 現在のテンポ(BPM)。初期値は120
33 long tickCount = 0; // 演奏開始からの累積tick数
34 unsigned long lastTickMs = 0; // 前回tickを打った時刻(ms)
```

```

35
36 int prevBtnOrder[NUM_CHILDREN] = {HIGH, HIGH, HIGH, HIGH}; // 参加ボタンの前回値(立
    下り検出用)
37 int prevBtnStart = HIGH; // 開始ボタンの前回値(立下り検出用)
38 int prevBtnReset = HIGH; // リセットボタンの前回値(立下り検出用)
39
40 // ポテンショメータ読み取りの平滑化と更新間引き用
41 unsigned long lastPotReadMs = 0; // 前回可変抵抗を読んだ時刻(ms)
42 int smoothedPotRaw = -1; // 平滑化後のADC生値(-1は未初期化)
43 int lastReportedBpm = -1; // 前回ログ出力したBPM(変化検出用)
44
45 void setup() { // 起動時に一度だけ実行される初期化関数
46     Serial.begin(115200); // PCとのシリアル通信を115200bpsで開始
47     IrSender.begin(IR_SEND_PIN); // 赤外線送信をD3ピンで初期化
48
49     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機分のピンを順に設定
50         pinMode(PIN_BTN_ORDER[i], INPUT_PULLUP); // 参加ボタンを内部プルアップ入力に設定
51         pinMode(PIN_LED_CHILD[i], OUTPUT); // 状態表示LEDを出力に設定
52         digitalWrite(PIN_LED_CHILD[i], LOW); // 起動時はLEDを消灯
53     }
54     pinMode(PIN_BTN_START, INPUT_PULLUP); // 開始ボタンをプルアップ入力に設定
55     pinMode(PIN_BTN_RESET, INPUT_PULLUP); // リセットボタンをプルアップ入力に設定
56     // PIN_POT_TEMPO はアナログ入力なので pinMode 不要 (pull-up は付けない)
57     pinMode(LED_INDICATOR, OUTPUT); // インジケータLEDを出力に設定
58     pinMode(PIN_SYNC_OUT, OUTPUT); // 同期パルス出力ピンを出力に設定
59     digitalWrite(PIN_SYNC_OUT, LOW); // 同期パルスを初期LOWにしておく
60
61     printHelp(); // 操作方法をシリアルに表示
62 }
63
64 void loop() { // メインループ(起動後ずっと繰り返す)
65     handleSerialCommand(); // PCからのシリアルコマンドを処理
66     handleOrderButtons(); // 各子機の参加/退場ボタンを処理
67     handleResetButton(); // リセットボタンを処理
68     handleTempoPot(); // 可変抵抗によるテンポ調整を処理
69     handleStartButton(); // 開始ボタンを処理
70
71     if (isPlaying) { // 演奏中のときだけtick処理を行う
72         unsigned long now = millis(); // 現在時刻(ms)を取得
73         unsigned long interval = 60000UL / (unsigned long)tempoBpm / 2UL; // 8分音符1つ
            分の時間間隔を計算
74         if (now - lastTickMs >= interval) { // 前回tickから間隔が経過したか判定
75             lastTickMs = now; // tick時刻を更新
76             tick(); // 1tick分の処理(マスク送信等)を実行
77             tickCount++; // 累積tick数を1増やす
78         }
79     }
80

```

```

81 // 子機LEDを内部状態に同期。isActive XOR (joinTick≠-1 || leaveTick≠-1) で
82 // 「参加意思あり=点灯」となり、ボタン押下毎に確実にトグルする。
83 // 4状態の遷移は tick() で join/leave 反映後も継続する:
84 // 未参加+予約なし(OFF) → 未参加+joinTick(ON) → tickでjoin → 参加+予約なし(ON)
85 // → 参加+leaveTick(OFF) → tickでleave → 未参加+予約なし(OFF)
86 for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機のLED状態を更新
87     bool on = isActive[i] != ((joinTick[i] != -1) || (leaveTick[i] != -1)); // 参加
        意思があれば点灯(XORでトグル表現)
88     digitalWrite(PIN_LED_CHILD[i], on ? HIGH : LOW); // 計算結果に従いLEDを点灯/消灯
89 }
90 }
91
92 // 参加ボタン (子機ごと) のトグル処理 (ボタン・シリアル共有)。
93 // 未参加 → 次の区切りから参加予約
94 // 参加予約中 → その予約を取消 (参加しない)
95 // 参加中 → 次の区切りから退場予約
96 // 退場予約中 → その予約を取消 (残留)
97 void toggleChild(int i) { // 子機iの参加/退場状態をトグルする
98     if (i < 0 || i >= NUM_CHILDREN) return; // 範囲外の子機番号は無視
99     if (!isPlaying) { // 停止中は操作を受け付けない
100         Serial.println("[!]_再生中のみ操作できます"); // 再生中のみ操作可能と警告表示
101         return; // 処理を中断
102     }
103
104     long joinBoundary = ((tickCount / QUANTIZE_TICKS) + 1) * QUANTIZE_TICKS; // 次の区
        切り(8tick境界)を参加tickに計算

105
106     if (isActive[i]) { // すでに参加中の場合
107         if (leaveTick[i] != -1) { // 退場予約済みなら
108             leaveTick[i] = -1; // 退場予約を取り消す
109             Serial.print("child_"); // ログ出力(子機番号の前置き)
110             Serial.print(i); // 子機番号を出力
111             Serial.println("_leave_cancelled_(stay)"); // 退場取消(残留)を通知
112         } else { // 退場予約がなければ新規に退場予約
113             // 子機が現在のループを最後まで弾き切ってから抜けるよう、
114             // joinedAtTick から数えて次の SCORE_TICKS 境界に揃える。
115             long ticksSinceJoin = tickCount - joinedAtTick[i]; // 参加してからの経過tick数
116             long loopsCompleted = ticksSinceJoin / SCORE_TICKS; // 完了した楽譜ループ数
117             leaveTick[i] = joinedAtTick[i] + (loopsCompleted + 1) * SCORE_TICKS; // 次の
                ループ末尾を退場tickに設定
118             Serial.print("child_"); // ログ出力
119             Serial.print(i); // 子機番号を出力
120             Serial.print("_will_leave_at_tick_"); // 退場予定tickの前置き
121             Serial.print(leaveTick[i]); // 退場予定tickを出力
122             Serial.print("_(end_of_loop_)"); // 何ループ目の末尾かの前置き
123             Serial.print(loopsCompleted + 1); // ループ番号を出力
124             Serial.println(""); // 行を閉じる

```

```

125     }
126 } else { // 未参加の場合
127     if (joinTick[i] != -1) { // 参加予約済みなら
128         joinTick[i] = -1; // 参加予約を取り消す
129         Serial.print("child_"); // ログ出力
130         Serial.print(i); // 子機番号を出力
131         Serial.println("_join_cancelled"); // 参加取消を通知
132     } else { // 参加予約がなければ新規に参加予約
133         joinTick[i] = joinBoundary; // 次の区切りを参加tickに設定
134         Serial.print("child_"); // ログ出力
135         Serial.print(i); // 子機番号を出力
136         Serial.print("_will_join_at_tick_"); // 参加予定tickの前置き
137         Serial.println(joinTick[i]); // 参加予定tickを出力
138     }
139 }
140 }
141
142 // 参加ボタン：押すごとに 参加予約⇔退場予約 をトグルする（リアルタイム、次の区切りに
    反映）
143 void handleOrderButtons() { // 参加/退場ボタンの押下を検出する関数
144     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機のボタンを順に確認
145         int s = digitalRead(PIN_BTN_ORDER[i]); // 現在のボタン状態を読む
146         if (prevBtnOrder[i] == HIGH && s == LOW) { // HIGH→LOWの立下り(押した瞬間)を検
            出
147             toggleChild(i); // 該当子機の状態をトグル
148             delay(30); // チャタリング防止に30ms待つ
149         }
150         prevBtnOrder[i] = s; // 今回の状態を次回比較用に保存
151     }
152 }
153
154 void doReset() { // 全状態を初期化して停止する関数
155     isPlaying = false; // 再生を停止
156     tickCount = 0; // tickカウンタを0に戻す
157     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機分の状態を初期化
158         isActive[i] = false; // 参加状態を解除
159         joinTick[i] = -1; // 参加予約をクリア
160         leaveTick[i] = -1; // 退場予約をクリア
161         joinedAtTick[i] = -1; // 参加発火tickをクリア
162     }
163     digitalWrite(LED_INDICATOR, LOW); // インジケータLEDを消灯
164     Serial.println("reset"); // リセット完了をログ出力
165 }
166
167 void handleResetButton() { // リセットボタンの押下を検出する関数
168     int s = digitalRead(PIN_BTN_RESET); // ボタン状態を読む
169     if (prevBtnReset == HIGH && s == LOW) { // 立下り(押下)を検出
170         doReset(); // リセット処理を実行

```

```

171     delay(30); // チャタリング防止
172 }
173 prevBtnReset = s; // 状態を保存
174 }
175
176 // 可変抵抗 (A5) の値を 60~150 BPM に線形マップしてリアルタイム反映する。
177 // 50ms間隔で読み取り、指数移動平均で軽くノイズを抑える。
178 // 既存の tick 周期計算は毎ループ tempoBpm を参照しているため、即座に反映される。
179 void handleTempoPot() { // 可変抵抗を読んでテンポに反映する関数
180     unsigned long now = millis(); // 現在時刻を取得
181     if (now - lastPotReadMs < POT_READ_INTERVAL_MS) return; // 前回から50ms未満なら読
        ますに戻る
182     lastPotReadMs = now; // 読み取り時刻を更新
183
184     int raw = analogRead(PIN_POT_TEMPO); // 可変抵抗のADC値(0~1023)を読む
185     if (smoothedPotRaw < 0) smoothedPotRaw = raw; // 初回はそのまま初期値にする
186     else smoothedPotRaw = (smoothedPotRaw * 3 + raw) / 4; // 指数移動平均でノイズを平
        滑化
187
188     int newBpm = map(smoothedPotRaw, 0, 1023, TEMPO_MIN_BPM, TEMPO_MAX_BPM); // ADC値
        を60~150BPMへ線形変換
189     newBpm = constrain(newBpm, TEMPO_MIN_BPM, TEMPO_MAX_BPM); // 範囲外に出ないようにク
        ランプ
190     tempoBpm = newBpm; // テンポを更新(次tickから即反映)
191
192     // 微小変動でシリアルが埋まらないよう、2BPM以上変わったときだけログ
193     if (lastReportedBpm < 0 || abs(newBpm - lastReportedBpm) >= 2) { // 2BPM以上変化し
        たときだけログ
194         Serial.print("tempo="); // テンポ値の前置きを出力
195         Serial.println(tempoBpm); // 現在のテンポを出力
196         lastReportedBpm = newBpm; // 報告済みBPMを更新
197     }
198 }
199
200 // 再生開始：誰も参加していない状態から開始する (参加はボタンで都度行う)
201 void doStart() { // 演奏を開始する関数
202     if (isPlaying) { // すでに再生中なら
203         Serial.println("[!]_already_playing"); // 二重開始を警告
204         return; // 何もせず戻る
205     }
206     isPlaying = true; // 再生中フラグを立てる
207     tickCount = 0; // tickカウンタを初期化
208     lastTickMs = millis() - (60000UL / (unsigned long)tempoBpm / 2UL); // すぐ最初の
        tickが出るよう時刻を過去にずらす
209     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機の状態を初期化
210         isActive[i] = false; // 参加状態を解除
211         joinTick[i] = -1; // 参加予約をクリア

```



```

212     leaveTick[i] = -1; // 退場予約をクリア
213     joinedAtTick[i] = -1; // 参加発火tickをクリア
214 }
215 Serial.println("start"); // 開始をログ出力
216 }
217
218 void handleStartButton() { // 開始ボタンの押下を検出する関数
219     int s = digitalRead(PIN_BTN_START); // ボタン状態を読む
220     if (prevBtnStart == HIGH && s == LOW) { // 立下り(押下)を検出
221         doStart(); // 演奏開始処理を実行
222         delay(30); // チャタリング防止
223     }
224     prevBtnStart = s; // 状態を保存
225 }
226
227 void doTest() { // 通信品質計測用のテスト送信関数
228     const long TEST_COUNT = 10000; // 送信回数(1万回)
229     const int SEND_INTERVAL_MS = 30; // 送信間隔(30ms)
230     Serial.println("[TEST]_start_10000_sends_(mask=0x0F,_interval=30ms)"); // テスト開
        始をログ出力
231     for (long i = 0; i < TEST_COUNT; i++) { // 1万回のループ
232         digitalWrite(PIN_SYNC_OUT, HIGH); // 同期パルスを立てる(計測起点)
233         delayMicroseconds(50); // 50us保持
234         digitalWrite(PIN_SYNC_OUT, LOW); // 同期パルスを下げる
235         IrSender.sendSony(0, 0x0F, 0); // 全子機宛(mask=0x0F)でSony信号を送信
236         if (i % 1000 == 0) { // 1000回ごとに
237             Serial.print("[TEST]_"); // 進捗の前置き
238             Serial.print(i); // 現在の送信回数
239             Serial.println("/10000"); // 総数を出力
240         }
241         delay(SEND_INTERVAL_MS); // 次の送信まで30ms待つ
242     }
243     Serial.println("[TEST]_done_10000/10000"); // テスト完了をログ出力
244 }
245
246 void handleSerialCommand() { // シリアル入力コマンドを処理する関数
247     while (Serial.available()) { // 受信バッファがある間繰り返す
248         char c = (char)Serial.read(); // 1文字読み取る
249         if (c == '\r' || c == '\n' || c == '_') continue; // 改行・空白は無視
250
251         if (c >= '0' && c <= ('0' + NUM_CHILDREN - 1)) { // '0'~'3'なら子機番号として扱
            う
252             toggleChild(c - '0'); // 対応子機をトグル
253         } else if (c == 's' || c == 'S') { // 's'なら
254             doStart(); // 演奏開始
255         } else if (c == 'r' || c == 'R') { // 'r'なら
256             doReset(); // リセット
257         } else if (c == 't' || c == 'T') { // 't'なら

```

```

258     doTest(); // テスト送信
259 } else if (c == '?') { // '?'なら
260     printHelp(); // ヘルプ表示
261 }
262 }
263 }
264
265 void printHelp() { // 操作方法をシリアルに表示する関数
266     Serial.println("===_hack_oya_kai_help_==="); // ヘルプの見出しを出力
267     Serial.println("0..3:_子機の参加/退場トグル_(D4..D7_ボタンと同じ動作)"); // 数字
        キーの説明
268     Serial.println("_____未参加→参加予約_/_参加中→退場予約_/_予約中に再度押すと取消
        "); // トグル遷移の説明
269     Serial.println("s_____:_start"); // sキーの説明
270     Serial.println("r_____:_reset_(即時の強制停止)"); // rキーの説明
271     Serial.println("?_____:_help"); // ?キーの説明
272     Serial.println("(BPMは_A5_のポテンショメータで_60-150_を連続調整)"); // テンポ調
        整方法の説明
273     Serial.print("Current_BPM="); // 現在BPMの前置き
274     Serial.println(tempoBpm); // 現在のテンポを出力
275 }
276
277 void tick() { // 1tickごとにマスクを送信する中核関数
278     digitalWrite(LED_INDICATOR, (tickCount % 2) ? HIGH : LOW); // tickごとにインジケー
        タLEDを点滅

279
280     uint8_t mask = 0; // 送信する参加楽器マスクを初期化
281     for (int i = 0; i < NUM_CHILDREN; i++) { // 全子機について
282         // 参加予約していたtickに到達したら有効化
283         if (!isActive[i] && joinTick[i] != -1 && tickCount >= joinTick[i]) { // 参加予約
            tickに到達したら
284             isActive[i] = true; // 参加状態にする
285             joinTick[i] = -1; // 参加予約をクリア
286             joinedAtTick[i] = tickCount; // 退場アライメント計算の起点
287             Serial.print("child_"); // ログ出力
288             Serial.print(i); // 子機番号を出力
289             Serial.println("_joined"); // 参加をログ出力
290         }
291         // 退場予約していたtickに到達したら無効化 (このtickからは送らない)
292         if (isActive[i] && leaveTick[i] != -1 && tickCount >= leaveTick[i]) { // 退場予
            約tickに到達したら
293             isActive[i] = false; // 参加状態を解除
294             leaveTick[i] = -1; // 退場予約をクリア
295             joinedAtTick[i] = -1; // 参加発火tickをクリア
296             Serial.print("child_"); // ログ出力
297             Serial.print(i); // 子機番号を出力
298             Serial.println("_left"); // 退場をログ出力

```

```

299     }
300     if (isActive[i]) { // 参加中の子機は
301         mask |= (uint8_t)(1 << i); // マスクの該当ビットを立てる
302     }
303 }
304
305 if (mask != 0) { // 参加中の子機が1つでもあれば
306     // 子機D3への同期パルス。IR送信直前に立てて、子機ISRのmicros()と
307     // 本ループのsendNEC完了までの遅延を子機側で計測できるようにする。
308     digitalWrite(PIN_SYNC_OUT, HIGH); // 送信直前に同期パルスを立てる(遅延計測用)
309     delayMicroseconds(50); // 50us保持
310     digitalWrite(PIN_SYNC_OUT, LOW); // 同期パルスを下げる
311
312     IrSender.sendSony(0, mask, 0); // 参加マスクをSony SIRCで全子機へ送信
313 }
314 }

```

B 子機 Arduino のスケッチ

ピアノ・フルート・トランペット・ドラムの4子機で共通の Arduino スケッチを以下に示す。書き込み時に childId を機体ごとに変更する。

```

1  #define DECODE_SONY // Sony SIRCプロトコルのデコードを有効化
2  #include <IRremote.hpp> // 赤外線受信ライブラリを読み込む
3
4  // hack_oya_kai.ino 用プロトコル対応版:
5  // IrSender.sendSony(0, mask, 0) を受信 (Sony SIRC 12bit, ~17ms)。
6  // mask の bit(childId) が立っているtickだけ「自分の出番」として
7  // localPos をカウントアップし、その位置の音をホストへ送信する。
8  // 親機側にループの概念がないが、メロディ機 (childId != DRUM_CHILD_ID)
9  // は楽譜の最後まで行ったら localPos を 0 に戻し、自力でループする。
10 // (ドラム機は元々楽譜を持たず鳴らし続けるだけなので、この対象外)
11
12 // 書き込む機体ごとに値を変えて Upload する: 0,1,2 = メロディ機, 3 = ドラム機。
13 int childId = 0; // 自分の担当ビット番号(機体ごとに0~3へ変更)
14 const int DRUM_CHILD_ID = 3; // ドラム機のID(3番はドラム扱い)
15 const int PIN_IR_RECV = 2; // 赤外線受光モジュールの入力ピン(D2)
16 const int PIN_SYNC_IN = 3; // 親機D9からの同期パルス入力(遅延計測用)
17 const int LED_INDICATOR = 13; // 動作確認用の内蔵LED(D13)
18
19 const int SCORE_LENGTH = 40; // 楽譜配列の要素数(音符数)
20 const String myScore[SCORE_LENGTH] = { // この子機が演奏する楽譜(音名の並び)
21     "C4", "D4", "E4", "F4", "E4", "D4", "C4", "R", // 1~8音目(冒頭ドレミファミレド)
22     "E4", "F4", "G4", "A4", "G4", "F4", "E4", "R", // 9~16音目(ミファソラソファミ)
23     "C4", "R", "C4", "R", "C4", "R", "C4", "R", // 17~24音目(ドを休符を挟んで4回)
24     "C4", "C4", "D4", "D4", "E4", "E4", "F4", "F4", // 25~32音目(各音2回、以降16分音
    符)

```

```

25  "E4", "R",  "D4", "R",  "C4", "R" , "R" , "R"  // 33~40音目(ミレドと休符で締め)
26  };
27
28  // index 24 以降は1IR=2音(16分音符相当)で発音する。
29  const int DOUBLE_START = 24;  // この位置以降は1tickで2音(16分音符)鳴らす
30
31  // 1周(ループ)に必要なtick数。
32  //   通常区間: 0 .. DOUBLE_START-1 (1tick=1音)
33  //   倍速区間: DOUBLE_START .. (1tick=2音)  なので tick数は半分になる
34  // 例) DOUBLE_START=24, SCORE_LENGTH=40 → 24 + (40-24)/2 = 32
35  const int TOTAL_TICKS = DOUBLE_START + (SCORE_LENGTH - DOUBLE_START) / 2;  // 楽譜1周
    に要するtick数(=32)

36
37  // Processing への通信フォーマット: "ピッチ名,duration秒,amplitude,localPos\n"
38  const float NOTE_DURATION_DEFAULT = 0.40f;  // 通常音符の長さ(秒)
39  const float NOTE_DURATION_HALF    = 0.20f;  // 倍速区間の音符長(秒)
40  const float NOTE_AMPLITUDE        = 0.60f;  // 音量(0~1)
41
42  // ドラム機が drum.pde に送るキック合図 (drum.pde は "C2" 単独行を期待)。
43  const String DRUM_NOTE = "C2";  // ドラム機がキックとして送る合図
44
45  // 自分の出番が来た回数 (= 親機のtickCountに相当)。受信したtickごとに進む。
46  long localPos = -1;  // -1 = まだ一度も自分の出番が来ていない
47
48  // テストモード: 't' で ON。受信カウント+レイテンシCSV出力のみ行う
49  bool testMode = false;  // テストモードのON/OFF
50  long testRecvCount = 0;  // テスト中の受信回数
51  unsigned long testLatencyMin = 0xFFFFFFFF;  // 遅延の最小値(初期値は最大に)
52  unsigned long testLatencyMax = 0;  // 遅延の最大値
53  unsigned long testLatencySum = 0;  // 遅延の合計(平均算出用)
54  unsigned long lastTestRecvMs = 0;  // 最後にテスト受信した時刻
55
56  unsigned long lastReceiveMs = 0;  // 前回IRを受信した時刻(ms)
57  unsigned long lastIntervalMs = 250;  // 双子音化区間の半tick遅延に使う
58
59  // 遅延計測: 親機の同期パルス立ち上がり時刻をISRで記録
60  volatile unsigned long syncRiseUs = 0;  // 同期パルス立ち上がり時刻(ISRで更新)
61
62  void onSyncRise() {  // 同期パルス割り込み時に呼ばれるISR
63    syncRiseUs = micros();  // 立ち上がり時刻をusで記録
64  }
65
66  void sendNoteToHost(const String& pitch, float duration, float amplitude, long pos) {
    // Processingへ音符情報を送る関数
67    Serial.print(pitch);  // 音名を送信
68    Serial.print(',');  // 区切りカンマ
69    Serial.print(duration, 3);  // 音の長さを小数3桁で送信

```

```

70 Serial.print(','); // 区切りカンマ
71 Serial.print(amplitude, 3); // 音量を小数3桁で送信
72 Serial.print(','); // 区切りカンマ
73 Serial.println(pos); // localPosを送信して改行
74 }
75
76 void printReady() { // 起動完了メッセージを出す関数
77     Serial.print("hack-ko_ready_CHILD_ID="); // 準備完了とID前置き
78     Serial.print(childId); // 自分のIDを出力
79     Serial.println(childId == DRUM_CHILD_ID ? "_ (DRUM)" : "_ (MELODY)"); // ドラム機か
        メロディ機かを表示
80 }
81
82 void setup() { // 起動時の初期化関数
83     Serial.begin(115200); // シリアル通信を115200bpsで開始
84
85     // UNO R4 等の native USB CDC は Serial.begin() 直後はホスト未接続のことがある。
86     // 最大3秒だけCDC接続完了を待つ (タイムアウトしてもそのまま進む)。
87     unsigned long boot_t = millis(); // 起動時刻を記録
88     while (!Serial && (millis() - boot_t) < 3000) { ; } // 最大3秒だけUSB接続を待つ
89
90     IrReceiver.begin(PIN_IR_RECV); // 赤外線受信をD2で開始
91     pinMode(PIN_SYNC_IN, INPUT); // 同期パルス入力ピンを入力に設定
92     attachInterrupt(digitalPinToInterrupt(PIN_SYNC_IN), onSyncRise, RISING); // 立ち上
        がりでISRを呼ぶよう割り込み設定
93     pinMode(LED_INDICATOR, OUTPUT); // インジケータLEDを出力に設定
94
95     delay(100); // 安定待ちに100ms
96     printReady(); // 起動完了を通知
97 }
98
99 void handleChildSerial() { // PCからのコマンド(テスト切替)を処理
100     while (Serial.available()) { // 受信がある間繰り返す
101         char c = (char)Serial.read(); // 1文字読む
102         if (c == '\r' || c == '\n' || c == '_') continue; // 改行・空白は無視
103         if (c == 't' || c == 'T') { // 't'ならテストモードを切替
104             testMode = !testMode; // ON/OFFを反転
105             testRecvCount = 0; // 受信カウントをリセット
106             testLatencyMin = 0xFFFFFFFF; // 最小遅延をリセット
107             testLatencyMax = 0; // 最大遅延をリセット
108             testLatencySum = 0; // 遅延合計をリセット
109             lastTestRecvMs = 0; // 最終受信時刻をリセット
110             if (testMode) { // テストON時は
111                 Serial.println("[TEST]_receive_mode_ON_—_waiting_for_IR..."); // 待機メッ
                    セージを出力
112                 Serial.println("seq,latency_us"); // CSVヘッダを出力
113             } else { // テストOFF時は
114                 Serial.println("[TEST]_receive_mode_OFF"); // 終了メッセージを出力

```

```

115     }
116 }
117 }
118 }
119
120 void loop() { // メインループ
121     handleChildSerial(); // シリアルコマンドを処理
122
123     // テスト完了検知: 最後の受信から3秒経ったらサマリーを出力して終了
124     if (testMode && testRecvCount > 0 && (millis() - lastTestRecvMs > 3000)) { // テス
        ト中で3秒受信が途絶えたら集計
125         Serial.println("---"); // 区切り線を出力
126         Serial.print("[TEST]_received:"); // 受信数の前置き
127         Serial.print(testRecvCount); // 受信回数を出力
128         Serial.println("/10000"); // 総数を出力
129         Serial.print("[TEST]_min="); // 最小遅延の前置き
130         Serial.print(testLatencyMin); // 最小遅延を出力
131         Serial.print("us_max="); // 最大遅延の前置き
132         Serial.print(testLatencyMax); // 最大遅延を出力
133         Serial.print("us_avg="); // 平均遅延の前置き
134         Serial.print(testLatencySum / testRecvCount); // 平均遅延を計算して出力
135         Serial.println("us"); // 単位を出力
136         long lost = 10000 - testRecvCount; // 取りこぼした回数を計算
137         Serial.print("[TEST]_lost:"); // ロスト数の前置き
138         Serial.println(lost); // ロスト数を出力
139         testMode = false; // テストモードを終了
140     }
141
142     if (!IrReceiver.decode()) return; // IR信号を受信していなければ戻る
143
144     // decode直後にデータを退避し、即座にresumeする。
145     // delay()中もIR受信を継続させ、tick取りこぼしを防ぐ。
146     uint8_t protocol = IrReceiver.decodedIRData.protocol; // 受信プロトコルを退避
147     uint8_t mask = (protocol == SONY) ? IrReceiver.decodedIRData.command : 0; // Sony
        ならコマンド(マスク)を取り出す
148     IrReceiver.resume(); // 次の受信に備えて即座に再開
149
150     if (protocol == SONY) { // 受信がSony信号のときだけ処理
151         if (testMode) { // テストモード中は計測だけ行う
152             testRecvCount++; // 受信回数を1増やす
153             lastTestRecvMs = millis(); // 最終受信時刻を更新
154             unsigned long recvUs = micros(); // 受信時刻をusで取得
155             unsigned long riseUs = syncRiseUs; // ISRが記録した同期時刻を取得
156             unsigned long lat = (riseUs != 0) ? (recvUs - riseUs) : 0; // 同期からの遅延を
                計算
157             if (lat > 0) { // 遅延が計測できたら
158                 if (lat < testLatencyMin) testLatencyMin = lat; // 最小値を更新
159                 if (lat > testLatencyMax) testLatencyMax = lat; // 最大値を更新

```

```

160     testLatencySum += lat; // 合計に加算
161 }
162 Serial.print(testRecvCount); // シーケンス番号を出力
163 Serial.print(','); // 区切りカンマ
164 Serial.println(lat); // 遅延値を出力(CSV)
165 return; // テスト中は発音せず戻る
166 }
167
168 if (mask & (1 << childId)) { // 自分の担当ビットが立っていれば出番
169     unsigned long recvUs = micros(); // 受信時刻を取得
170     unsigned long riseUs = syncRiseUs; // ISRで記録された値をコピー
171     Serial.print("[RECV]_millis="); // 受信ログの前置き
172     Serial.print(millis()); // 受信時刻(ms)を出力
173     Serial.print("_child="); // 子機番号の前置き
174     Serial.print(childId); // 自分のIDを出力
175     Serial.print("_localPos="); // 楽譜位置の前置き
176     Serial.print(localPos + 1); // 次の楽譜位置を出力
177     if (riseUs != 0) { // 同期時刻が取れていれば
178         Serial.print("_latency="); // 遅延の前置き
179         Serial.print(recvUs - riseUs); // 遅延値を出力
180         Serial.print("us"); // 単位を出力
181     }
182     Serial.println(); // ログ行を閉じる
183     localPos++; // 楽譜位置を1進める
184
185     if (childId == DRUM_CHILD_ID) { // ドラム機の場合
186         Serial.println(DRUM_NOTE); // キック合図(C2)を送信
187         digitalWrite(LED_INDICATOR, HIGH); // LEDを点灯
188         delay(15); // 15ms点灯
189         digitalWrite(LED_INDICATOR, LOW); // LEDを消灯
190     } else { // メロディ機の場合
191         if (localPos >= TOTAL_TICKS) { // 楽譜末尾を超えたら
192             localPos = 0; // 先頭へ戻り自力でループ
193         }
194
195         unsigned long now = millis(); // 現在時刻を取得
196         if (lastReceiveMs != 0) { // 前回受信時刻があれば
197             unsigned long interval = now - lastReceiveMs; // tick間隔を計算
198             // 演奏停止→再開時の大きなギャップでdelayが膨張するのを防ぐ
199             if (interval >= 50 && interval <= 2000 && interval <= lastIntervalMs * 3) {
200                 // 妥当な間隔のときだけ採用
201                 lastIntervalMs = interval; // 直近のtick間隔を更新
202             }
203         }
204         lastReceiveMs = now; // 受信時刻を保存
205
206         if (localPos >= 0 && localPos < DOUBLE_START) { // 通常区間(1tick=1音)なら
207             sendNoteToHost(myScore[localPos], NOTE_DURATION_DEFAULT, NOTE_AMPLITUDE,

```

```

207         localPos); // 該当音符をProcessingへ送信
        digitalWrite(LED_INDICATOR, (localPos % 2) ? HIGH : LOW); // 位置に応じLED
            を点滅
208    } else if (localPos >= DOUBLE_START) { // 倍速区間(1tick=2音)なら
209        int scoreIdx = DOUBLE_START + 2 * (int)(localPos - DOUBLE_START); // 楽譜
            配列の実インデックスを算出
210        if (scoreIdx < SCORE_LENGTH) { // 範囲内なら1音目を
211            sendNoteToHost(myScore[scoreIdx], NOTE_DURATION_HALF, NOTE_AMPLITUDE,
                localPos); // 1音目を送信
212            digitalWrite(LED_INDICATOR, (localPos % 2) ? HIGH : LOW); // LEDを点滅
213        }
214        if (scoreIdx + 1 < SCORE_LENGTH) { // 2音目が範囲内なら
215            unsigned long halfDelay = min(lastIntervalMs / 2, 200UL); // 半tick分の
                待ち時間を計算
216            delay(halfDelay); // 半tick待つ
217            sendNoteToHost(myScore[scoreIdx + 1], NOTE_DURATION_HALF, NOTE_AMPLITUDE,
                localPos); // 2音目を送信
218        }
219    }
220 }
221 }
222 }
223 }

```

C ピアノ音再生用 Processing プログラム

ピアノ音の合成と再生を行う Processing のプログラムを以下に示す。

```

1  import processing.serial.*; // シリアル通信ライブラリを読み込む
2  Serial port; // Arduinoと通信するシリアルポート
3  import ddf.minim.*; // 音声合成ライブラリMinimを読み込む
4  import ddf.minim.ugens.*; // Minimの音源生成ユニット群を読み込む
5  import java.util.HashMap; // キャッシュ用のHashMapを読み込む
6  Minim minim; // Minim本体のインスタンス
7  AudioOutput out; // 音声出力先
8  String currentNote = ""; // 現在表示中の音名(画面表示用)
9  int currentPos = -1; // 親機が割り当ててきた localPos (デバッグ表示用)
10
11 // ---- グランドピアノ音源 (音源部のみ差し替え。シリアル受信の仕組みは既存のまま)
    ----
12 // hack_koP2saisei_pianotest.pde と同一の合成アルゴリズム・同一パラメータ (音響検証7
    /7合格)。
13 // 方式: 既知ピッチは起動時にプリレンダしてSamplerにキャッシュ、発音はtrigger()のみ。
14 HashMap<String, Sampler> pianoCache = new HashMap<String, Sampler>(); // 生成済みピ
    アノ音のキャッシュ
15 Sampler lastSampler = null; // 直前に鳴らした音。R受信時にstop()で即打ち切る (旧
    noteOff()と同じ役割)

```



```

16 final float SAMPLE_RATE = 44100f; // サンプリング周波数(44.1kHz)
17 final float RELEASE_TAIL = 0.8f; // ノート長に加えてレンダリングする余韻(秒)
18
19 void setup() { // 起動時の初期化関数
20     size(512, 200); // 波形表示ウィンドウのサイズ
21     minim = new Minim(this); // Minimを初期化
22     out = minim.getLineOut(); // ライン出力を取得
23
24     // 楽譜(hack-koP2.ino)で使うピッチ×チ2種の長さを起動時にプリレンダリング。
25     // 未知のピッチ/長さ/音量が来ても playNote 側の安全網で都度生成される。
26     // ポートを開く前に済ませ、プリレンダ中のシリアル取りこぼしを避ける。
27     String[] knownPitches = { "C4", "D4", "E4", "F4", "G4", "A4" }; // 事前生成する音
        名の一覧
28     float[] knownDurs = { 0.40f, 0.20f }; // 事前生成する音の長さ2種
29     long t0 = millis(); // プリレンダ開始時刻を記録
30     println("ピアノ音源をプリレンダリング中..."); // 進捗メッセージを表示
31     for (String pitch : knownPitches) { // 各音名について
32         for (float dur : knownDurs) { // 各長さについて
33             String key = cacheKey(pitch, dur, 0.60f); // キャッシュキーを生成
34             pianoCache.put(key, buildSampler(pitch, dur, 0.60f)); // 音を生成してキャッ
                シュに登録
35             println("_" + key + "_OK"); // 生成完了を表示
36         }
37     }
38     println("プリレンダ完了:" + pianoCache.size() + "音_" + (millis() - t0) + "ms");
        // 生成数と所要時間を表示

39
40     // portは各自変えて。
41     port = new Serial(this, "/dev/cu.usbserial-A5069RR4", 115200); // シリアルポートを
        開く
42     port.clear(); // 受信バッファをクリア
43     port.bufferUntil('\n'); // 改行までを1メッセージとして受信
44 }
45
46 void draw() { // 毎フレームの描画関数
47     background(0); // 背景を黒で塗る
48     stroke(255); // 描画色を白に
49     for (int i = 0; i < out.bufferSize() - 1; i++) { // 出力バッファの各サンプルについ
        て
50         line(i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50); // 波形を線で
            描画
51     }
52     fill(255); // 文字色を白に
53     text("note:" + currentNote, 10, 170); // 現在の音名を表示
54     text("pos:" + currentPos, 10, 190); // 現在の楽譜位置を表示
55 }
56

```

```

57 void serialEvent(Serial p) { // シリアル受信時に呼ばれる関数
58     String inString = p.readStringUntil('\n'); // 1行読み取る
59     if (inString == null) return; // 空なら戻る
60     inString = trim(inString); // 前後の空白を除去
61     if (inString.length() == 0) return; // 空行なら戻る
62     currentNote = inString; // 受信文字列を表示用に保存
63
64     // ino側のフォーマット: "ピッチ名,duration秒,amplitude,localPos"
65     // localPos は親機が決めた楽譜index。発音には使わずデバッグ表示専用。
66     // 旧3列フォーマット(pitch,duration,amplitude)も互換のため受け付ける。
67     // それ以外(起動メッセージなど)はパース不能としてスキップ。
68     String[] parts = split(inString, ','); // カンマ区切りで分解
69     if (parts.length < 3) { // 3要素未満なら音符でない
70         println("non-note:_" + inString); // 音符以外としてログ
71         return; // 戻る
72     }
73     String pitch = parts[0]; // 1列目=音名
74     float duration = float(parts[1]); // 2列目=長さ(秒)
75     float amplitude = float(parts[2]); // 3列目=音量
76     if (parts.length >= 4) { // 4列目があれば
77         currentPos = int(parts[3]); // localPosを表示用に保存
78     }
79     // R は休符: 直前の音の余韻が次のtickまで残らないよう、即stop()で打ち切る。
80     // (NOTE_DURATION_DEFAULT=0.40s は tick間隔(120BPMで0.25s)より長いので、
81     // R が来た時点で前音がまだ鳴っているのを止めないと休符に聞こえない。)
82     if (pitch.equals("R")) { // Rは休符なので
83         if (lastSampler != null) { // 直前の音が鳴っていれば
84             lastSampler.stop(); // 即座に止めて余韻を切る
85             lastSampler = null; // 参照をクリア
86         }
87         return; // 発音せず戻る
88     }
89     playNote(pitch, duration, amplitude); // 音名に対応する音を再生
90 }
91
92 void playNote(String pitch, float duration, float amplitude) { // 指定の音を鳴らす関
  数
93     String key = cacheKey(pitch, duration, amplitude); // キャッシュキーを生成
94     Sampler smp = pianoCache.get(key); // キャッシュから音を取得
95     if (smp == null) { // 安全網: 未キャッシュの組み合わせは都度生成してキャッシュ
96         smp = buildSampler(pitch, duration, amplitude); // 未生成なら新たに合成
97         pianoCache.put(key, smp); // キャッシュに登録
98     }
99     smp.trigger(); // 音を発音
100     lastSampler = smp; // 直近の音として保存
101 }
102
103 String cacheKey(String pitch, float duration, float amplitude) { // キャッシュキー文

```

```

104     字列を作る関数
105     return pitch + "_" + nf(duration, 0, 2) + "_" + nf(amplitude, 0, 2); // 音名・長
106     さ・音量を連結
107 }
108 // ---- グランドピアノ音源 (pianotestから無改変移植。amplitudeのみ引数化) -----
109 Sampler buildSampler(String pitch, float dur, float amp) { // ピアノ音を合成して
110     Samplerにする関数
111     float f0 = Frequency.ofPitch(pitch).asHz(); // 音名から基本周波数を求める
112     float[] data = renderPianoNote(f0, dur, amp); // 波形データを生成
113     MultiChannelBuffer mcb = new MultiChannelBuffer(data.length, 1); // 1chのバッファ
114     を用意
115     mcb.setChannel(0, data); // 波形データを書き込む
116     Sampler smp = new Sampler(mcb, SAMPLE_RATE, 6); // 最大6声 (余韻の重なり用)
117     smp.patch(out); // 出力に接続
118     return smp; // 生成したSamplerを返す
119 }
120 float[] renderPianoNote(float f0, float dur, float amp) { // ピアノ波形を数値計算で
121     生成する関数
122     int n = (int)((dur + RELEASE_TAIL) * SAMPLE_RATE); // 余韻を含む総サンプル数
123     float[] buf = new float[n]; // 波形バッファを確保
124
125     // 複弦構成 (実ピアノ: 高音ほど弦が多い)。デチューンはcent単位
126     float[] cents; // 各弦のデチューン量(cent)
127     if (f0 >= 350f) cents = new float[] { 0f, 1.2f, -0.8f }; // 高音域は3本弦
128     else if (f0 >= 150f) cents = new float[] { 0f, 1.0f }; // 中音域は2本弦
129     else cents = new float[] { 0f }; // 低音域は1本弦
130
131     float p = 0.9f + 0.5f * constrain((f0 - 110f) / 770f, 0f, 1f); // 倍音ロール
132     オフ
133     float B = 0.0002f * pow(f0 / 261.6f, 0.8f); // インハーモ
134     ニシティ
135     float T60 = constrain(6.0f * pow(261.6f / f0, 0.7f), 0.8f, 8.0f); // 残響長(音
136     域依存)
137     float tauBase = T60 / 6.9f; // 基準となる減衰時定数
138     float stringAmp = amp / cents.length; // 弦数で割った1本あたりの振幅
139
140     for (int s = 0; s < cents.length; s++) { // 各弦について
141         float fs = f0 * pow(2f, cents[s] / 1200f); // デチューンを反映した弦の周波数
142         for (int h = 1; h <= 10; h++) { // 第1~10倍音について
143             float fh = h * fs * sqrt(1f + B * h * h); // インハーモニシティ補正周波数
144             if (fh > SAMPLE_RATE * 0.45f) break; // ナイキスト付近を超える倍音は打ち切り
145             float ah = stringAmp / pow(h, p); // 倍音次数に応じ振幅を減衰
146             float tauH = tauBase / (1f + 0.6f * (h - 1)); // 高次倍音ほど速く減衰
147             float w = TWO_PI * fh / SAMPLE_RATE; // 1サンプルあたりの位相増分
148             for (int i = 0; i < n; i++) { // 全サンプルについて
149                 float t = i / SAMPLE_RATE; // 経過時間(秒)

```

```

144     float env = 0.3f * exp(-t / 0.12f) + 0.7f * exp(-t / tauH); // 2段階減衰
145     buf[i] += ah * env * sin(w * i); // 倍音成分を波形に加算
146 }
147 }
148 }
149
150 // ハンマーノイズ: 最初の15ms、1次LPF(fc=f0*8上限8k)、指数減衰
151 float fc = min(f0 * 8f, 8000f); // ノイズLPFのカットオフ周波数
152 float a = 1f - exp(-TWO_PI * fc / SAMPLE_RATE); // 1次LPFの係数
153 float lp = 0f; // LPFの状態変数
154 int nNoise = (int)(0.015f * SAMPLE_RATE); // ノイズを乗せる長さ(15ms分)
155 randomSeed(12345); // 打鍵音の再現性
156 for (int i = 0; i < nNoise && i < n; i++) { // ノイズ区間の各サンプルについて
157     float t = i / SAMPLE_RATE; // 経過時間
158     float x = random(-1f, 1f); // 白色ノイズを生成
159     lp += a * (x - lp); // LPFを通す
160     buf[i] += amp * 0.15f * lp * exp(-t / 0.008f); // 減衰させて打鍵音として加算
161 }
162
163 // アタック(3ms)とダンパーリリース(dur経過後  $\tau=0.15s$ )
164 for (int i = 0; i < n; i++) { // 全サンプルについて
165     float t = i / SAMPLE_RATE; // 経過時間
166     buf[i] *= (1f - exp(-t / 0.003f)); // 3msのソフトアタックを掛ける
167     if (t > dur) buf[i] *= exp(-(t - dur) / 0.15f); // 音符長経過後はダンパーで減衰
168 }
169
170 // クリップ防止
171 float peak = 0f; // 最大振幅を求める変数
172 for (int i = 0; i < n; i++) peak = max(peak, abs(buf[i])); // 波形のピークを探索
173 if (peak > 0.98f) { // ピークが0.98を超えたら
174     float g = 0.98f / peak; // 収める倍率を計算
175     for (int i = 0; i < n; i++) buf[i] *= g; // 全体を縮小してクリップ防止
176 }
177 return buf; // 完成した波形を返す
178 }

```

D フルート音再生用 Processing プログラム

フルート音の合成と再生を行う Processing のプログラムを以下に示す。

```

1 import processing.serial.*; // シリアル通信ライブラリを読み込む
2 Serial port; // シリアルポート
3 import ddf.minim.*; // 音声合成ライブラリMinim
4 import ddf.minim.ugens.*; // Minimの音源ユニット群
5 Waveform currentWaveform; // フルート用の波形テーブル
6 Minim minim; // Minim本体
7 AudioOutput out; // 音声出力先

```

```

8 String currentNote = ""; // 現在の音名(表示用)
9 int currentPos = -1; // 親機が割り当ててきた localPos (デバッグ表示用)
10
11 void setup() { // 起動時の初期化関数
12     size(512, 200); // 波形表示ウィンドウのサイズ
13     minim = new Minim(this); // Minimを初期化
14     out = minim.getLineOut(); // ライン出力を取得
15     out.setTempo(120); // 基準テンポを120に設定
16     // フルーツは基音が支配的で高次倍音が弱い。第2倍音を少しだけ加える。
17     currentWaveform = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.18f, 0.06f,
18         0.02f }); // 基音中心の倍音構成で波形を生成
19     // portは各自変えて。
20     port = new Serial(this, "/dev/cu.usbmodem64E8335D22342", 115200); // シリアルポ
21     // ートを開く
22     port.clear(); // 受信バッファをクリア
23     port.bufferUntil('\n'); // 改行までを1メッセージとして受信
24 }
25
26 void draw() { // 毎フレームの描画関数
27     background(0); // 背景を黒で塗る
28     stroke(255); // 描画色を白に
29     for (int i = 0; i < out.bufferSize() - 1; i++) { // 出力バッファの各サンプルについ
30         て
31         line(i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50); // 波形を線で
32         描画
33     }
34     fill(255); // 文字色を白に
35     text("note:_ " + currentNote, 10, 170); // 現在の音名を表示
36     text("pos:_ " + currentPos, 10, 190); // 現在の楽譜位置を表示
37 }
38
39 void serialEvent(Serial p) { // シリアル受信時に呼ばれる関数
40     String inString = p.readStringUntil('\n'); // 1行読み取る
41     if (inString == null) return; // 空なら戻る
42     inString = trim(inString); // 前後の空白を除去
43     if (inString.length() == 0) return; // 空行なら戻る
44     currentNote = inString; // 表示用に保存
45
46     // ino側のフォーマット: "ピッチ名,duration秒,amplitude,localPos"
47     // localPos は親機が決めた楽譜index。発音には使わずデバッグ表示専用。
48     // 旧3列フォーマット(pitch,duration,amplitude)も互換のため受け付ける。
49     // それ以外(起動メッセージなど)はパース不能としてスキップ。
50     String[] parts = split(inString, ','); // カンマ区切りで分解
51     if (parts.length < 3) { // 3要素未満なら音符でない
52         println("non-note:_ " + inString); // 音符以外としてログ
53         return; // 戻る
54     }
55     String pitch = parts[0]; // 1列目=音名

```

```

52 float duration = float(parts[1]); // 2列目=長さ(秒)
53 float amplitude = float(parts[2]); // 3列目=音量
54 if (parts.length >= 4) { // 4列目があれば
55     currentPos = int(parts[3]); // localPosを表示用に保存
56 }
57 if (pitch.equals("R")) return; // 休符なら発音せず戻る
58 playNote(pitch, duration, amplitude); // 音を再生
59 }
60
61 void playNote(String pitch, float duration, float amplitude) { // 指定の音を鳴らす関
    数
62     float freq = Frequency.ofPitch(pitch).asHz(); // 音名から周波数を求める
63     out.playNote(0, duration, new FluteInstrument(freq, amplitude, currentWaveform));
        // フルート楽器を生成して演奏
64 }
65
66 class FluteInstrument implements Instrument { // フルート音を定義する楽器クラス
67     Oscil wave; // 主音を出す発振器
68     ADSR ampEnv; // 振幅のエンベロープ
69     Oscil vib; // ビブラート用の発振器
70     Summer freqSum; // 基音とビブラートを合成する加算器
71     Constant baseFreq; // 基準周波数を出す定数源
72     float maxAmp; // 最大振幅
73
74     FluteInstrument(float frequency, float maxAmp, Waveform wf) { // コンストラクタ(音
        を組み立てる)
75         this.maxAmp = maxAmp; // 最大振幅を保存
76         freqSum = new Summer(); // 周波数加算器を生成
77         baseFreq = new Constant(frequency); // 基準周波数を設定
78         // フルートらしい穏やかなビブラート (約±5Hz、周波数の深さ)。
79         vib = new Oscil(5, frequency * 0.006f, Waves.SINE); // 約5Hzのビブラートを生成
80         baseFreq.patch(freqSum); // 基準周波数を加算器へ
81         vib.patch(freqSum); // ビブラートを加算器へ
82         // 振幅はADSR側でかけるので、ここでは 1.0 にしておく (0だと無音になる)。
83         wave = new Oscil(frequency, 1.0f, wf); // 主発振器を生成(振幅は後段で制御)
84         freqSum.patch(wave.frequency); // 合成周波数を発振器へ入力
85         // ピアノ的な瞬時アタック→減衰ではなく、ソフトに立ち上がり保持するエンベロープ。
86         // ADSR(maxAmp, attack, decay, sustainLevel, release)
87         ampEnv = new ADSR(this.maxAmp, 0.08f, 0.05f, this.maxAmp * 0.9f, 0.12f); // 柔ら
            かい立ち上りのADSRを設定
88         wave.patch(ampEnv); // 発振器をエンベロープへ接続
89     }
90
91     void noteOn(float duration) { // 発音開始時の処理
92         ampEnv.noteOn(); // エンベロープを立ち上げる
93         ampEnv.patch(out); // 出力に接続
94     }
95

```

```

96 void noteOff() { // 発音終了時の処理
97     ampEnv.noteOff(); // エンベロープを解放
98     ampEnv.unpatchAfterRelease(out); // 減衰後に出力から切断
99 }
100 }

```

E トランペット音再生用 Processing プログラム

トランペット音の合成と再生を行う Processing のプログラムを以下に示す。

```

1 import processing.serial.*; // シリアル通信ライブラリを読み込む
2 Serial port; // シリアルポート
3 import ddf.minim.*; // 音声合成ライブラリ Minim
4 import ddf.minim.ugens.*; // Minimの音源ユニット群
5
6 // 1. 変数の宣言
7 Waveform lowWave; // 低音域用の波形テーブル
8 Waveform midWave; // 中音域用の波形テーブル
9 Waveform highWave; // 高音域用の波形テーブル
10
11 Minim minim; // Minim本体
12 AudioOutput out; // 音声出力先
13 String currentNote = ""; // 現在の音名(表示用)
14
15 void setup() { // 起動時の初期化関数
16     size(512, 200); // 波形表示ウィンドウのサイズ
17     pixelDensity(1); // ピクセル密度を1に固定
18     minim = new Minim(this); // Minimを初期化
19     out = minim.getLineOut(); // ライン出力を取得
20     out.setTempo(120); // 基準テンポを120に設定
21
22
23     lowWave = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.72f, 0.5f, 0.3f,
24         0.1f }); // 低音域は倍音を豊かに
25     midWave = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.43f, 0.3f, 0.15f,
26         0.05f }); // 中音域は中程度の倍音
27     highWave = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.028f, 0.01f });
28     // 高音域はほぼ基音のみ
29
30
31 // シリアルポート設定
32 port = new Serial(this, "/dev/cu.usbmodem64E83364FFA82", 115200); // シリアルポ
    トを開く
33 port.clear(); // 受信バッファをクリア
34 port.bufferUntil('\n'); // 改行までを1メッセージとして受信
35 }
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

33 void draw() { // 毎フレームの描画関数
34     background(0); // 背景を黒で塗る
35     stroke(255); // 描画色を白に
36     for (int i = 0; i < out.bufferSize() - 1; i++) { // 出力バッファの各サンプルについ
        て
37         line(i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50); // 波形を線で
            描画
38     }
39     fill(255); // 文字色を白に
40     text("note:_" + currentNote, 10, 180); // 現在の音名を表示
41 }
42
43 void serialEvent(Serial p) { // シリアル受信時に呼ばれる関数
44     String inString = p.readStringUntil('\n'); // 1行読み取る
45     if (inString == null) return; // 空なら戻る
46     inString = trim(inString); // 前後の空白を除去
47     if (inString.length() == 0) return; // 空行なら戻る
48     currentNote = inString; // 表示用に保存
49
50     String[] parts = split(inString, ','); // カンマ区切りで分解
51
52     if (parts.length < 3) { // 3要素未満なら音符でない
53         if (inString.equals("C2")) { // C2ならドラムのキック合図
54             println("🥁_ドラム音(C2)を受信"); // 受信をログ表示
55         }
56         return; // 音符でないので戻る
57     }
58
59     String pitch = parts[0]; // 1列目=音名
60     if (pitch.equals("R")) { // 休符なら
61         println("♪_[_休符]"); // 休符をログ表示
62         return; // 発音せず戻る
63     }
64
65     float duration = float(parts[1]); // 2列目=長さ(秒)
66     float amplitude = float(parts[2]); // 3列目=音量
67
68     playNote(pitch, duration, amplitude); // 音を再生
69 }
70
71 void playNote(String pitch, float duration, float amplitude) { // 指定の音を鳴らす関
    数
72     try { // 不正な音名に備えて例外処理
73         float freq = Frequency.ofPitch(pitch).asHz(); // 音名から周波数を求める
74
75         // 3. 周波数に応じて3つのウェーブテーブルの値を正しく出し分け
76         Waveform selectedWave = midWave; // 既定は中音域波形
77         if (freq < 550) selectedWave = lowWave; // 550Hz未満は低音域波形

```



```

78     else if (freq > 900) selectedWave = highWave; // 900Hz超は高音域波形
79
80     out.playNote(0, duration, new TrumpetInstrument(freq, amplitude, selectedWave));
81     // トランペット楽器で演奏
82 } catch (Exception e) { // 例外がでてく
83     // 安全弁
84 }
85
86 class TrumpetInstrument implements Instrument { // トランペット音を定義する楽器クラ
87     // ス
88     Oscil wave; // 主音を出す発振器
89     Line ampEnv; // 振幅の直線エンベロープ
90     Oscil vib; // ビブラート用の発振器
91     Summer freqSum; // 基音とビブラートを合成する加算器
92     Constant baseFreq; // 基準周波数を出す定数源
93     float maxAmp; // 最大振幅
94
95     TrumpetInstrument(float frequency, float maxAmp, Waveform wf) { // コンストラクタ(
96         // 音を組み立てる)
97         this.maxAmp = maxAmp; // 最大振幅を保存
98         freqSum = new Summer(); // 周波数加算器を生成
99         baseFreq = new Constant(frequency); // 基準周波数を設定
100
101         vib = new Oscil(5.5f, 3.0f, Waves.SINE); // 5.5Hz・深さ3Hzのビブラート
102         baseFreq.patch(freqSum); // 基準周波数を加算器へ
103         vib.patch(freqSum); // ビブラートを加算器へ
104
105         wave = new Oscil(frequency, 0, wf); // 主発振器(振幅0で開始)
106         freqSum.patch(wave.frequency); // 合成周波数を発振器へ入力
107
108         ampEnv = new Line(); // 振幅制御用の直線を生成
109         ampEnv.patch(wave.amplitude); // 発振器の振幅に接続
110     }
111
112     void noteOn(float duration) { // 発音開始時の処理
113         float attackTime = 0.05f; // 立上り時間(50ms)
114         ampEnv.activate(attackTime, 0, this.maxAmp); // 0から最大振幅へ立ち上げる
115         wave.patch(out); // 発振器を出力に接続
116     }
117
118     void noteOff() { // 発音終了時の処理
119         float fadeOutTime = 0.03f; // フェードアウト時間(30ms)
120         ampEnv.activate(fadeOutTime, this.maxAmp, 0); // 最大振幅から0へ下げる
121
122         // 音が消えるのを待ってから安全に切断
123         out.setNoteOffset(fadeOutTime); // フェードアウト分だけ切断を遅らせる
124         wave.unpatch(out); // 発振器を出力から切断

```

```
123 }
124 }
```

F ドラム音再生用 Processing プログラム

ドラム音の合成と再生を行う Processing のプログラムを以下に示す。

```
1 import ddf.minim.*; // 音声合成ライブラリ Minim
2 import ddf.minim.ugens.*; // Minimの音源ユニット群
3 import processing.serial.*; // シリアル通信ライブラリ
4
5 Minim minim; // Minim本体
6 AudioOutput out; // 音声出力先
7 Serial arduino; // Arduinoと通信するシリアルポート
8
9 // =====
10 // シリアル設定
11 // =====
12
13 final int SERIAL_PORT_INDEX = 0; // 環境に合わせて変更
14 final int BAUD_RATE = 115200; // シリアル通信速度
15
16 // =====
17 // キック
18 // =====
19
20 Oscil kickOsc; // キックの基音発振器
21 Multiplier kickAmp; // キック基音の音量制御
22
23 Oscil kickHarmonic; // キックの倍音発振器
24 Multiplier kickHarmonicAmp; // キック倍音の音量制御
25
26 // =====
27 // シンバル
28 // =====
29
30 Noise cymNoise; // シンバルのノイズ源
31 Multiplier cymNoiseAmp; // シンバルノイズの音量制御
32
33 Oscil cymMetal1; // シンバル金属倍音1
34 Oscil cymMetal2; // シンバル金属倍音2
35 Oscil cymMetal3; // シンバル金属倍音3
36
37 Multiplier cymMetalAmp1; // 金属倍音1の音量制御
38 Multiplier cymMetalAmp2; // 金属倍音2の音量制御
39 Multiplier cymMetalAmp3; // 金属倍音3の音量制御
40
```

```

41 // =====
42 // 自動演奏
43 // =====
44
45 boolean autoPlay = false; // 自動演奏のON/OFF
46 int lastTriggerTime = 0; // 前回自動発音した時刻
47 int autoStep = 0; // 自動演奏のステップ位置
48
49 // =====
50 // 初期設定
51 // =====
52
53 void setup() { // 起動時の初期化関数
54     size(700, 260); // ウィンドウサイズ
55
56     minim = new Minim(this); // Minimを初期化
57     out = minim.getLineOut(Minim.MONO, 512); // モノラルのライン出力を取得
58
59     setupKick(); // キック音源を構築
60     setupCymbal(); // シンバル音源を構築
61     setupSerial(); // シリアル通信を初期化
62
63     textSize(18); // 画面文字サイズを設定
64 }
65
66 void setupKick() { // キック音源を組み立てる関数
67     kickOsc = new Oscil(48.0, 1.0, Waves.SINE); // 48Hzの正弦波で基音を作る
68     kickAmp = new Multiplier(0.0); // 音量制御(初期は無音)
69     kickOsc.patch(kickAmp).patch(out); // 発振器→音量→出力へ接続
70
71     kickHarmonic = new Oscil(96.0, 1.0, Waves.SINE); // 96Hz(倍音)の発振器
72     kickHarmonicAmp = new Multiplier(0.0); // 倍音の音量制御
73     kickHarmonic.patch(kickHarmonicAmp).patch(out); // 倍音→音量→出力へ接続
74 }
75
76 void setupCymbal() { // シンバル音源を組み立てる関数
77     cymNoise = new Noise(1.0, Noise.Tint.WHITE); // 白色ノイズを生成
78     cymNoiseAmp = new Multiplier(0.0); // ノイズの音量制御
79     cymNoise.patch(cymNoiseAmp).patch(out); // ノイズ→音量→出力へ接続
80
81     cymMetal1 = new Oscil(431.0, 1.0, Waves.SQUARE); // 431Hzの矩形波(金属倍音1)
82     cymMetal2 = new Oscil(647.0, 1.0, Waves.SQUARE); // 647Hzの矩形波(金属倍音2)
83     cymMetal3 = new Oscil(913.0, 1.0, Waves.SQUARE); // 913Hzの矩形波(金属倍音3)
84
85     cymMetalAmp1 = new Multiplier(0.0); // 金属倍音1の音量制御
86     cymMetalAmp2 = new Multiplier(0.0); // 金属倍音2の音量制御
87     cymMetalAmp3 = new Multiplier(0.0); // 金属倍音3の音量制御
88

```

```

89   cymMetal1.patch(cymMetalAmp1).patch(out); // 倍音1→音量→出力へ接続
90   cymMetal2.patch(cymMetalAmp2).patch(out); // 倍音2→音量→出力へ接続
91   cymMetal3.patch(cymMetalAmp3).patch(out); // 倍音3→音量→出力へ接続
92 }
93
94 void setupSerial() { // シリアル通信を初期化する関数
95   println("===_Serial_Ports_==="); // ポート一覧の見出しを表示
96   printArray(Serial.list()); // 利用可能なポートを列挙
97
98   final String SERIAL_PORT_NAME = "/dev/cu.usbmodem34B7DA64C6002"; // 使用するポート
      名
99
100   arduino = new Serial(this, SERIAL_PORT_NAME, BAUD_RATE); // シリアルポートを開く
101   arduino.bufferUntil('\n'); // 改行までを1メッセージとして受信
102 }
103
104 // =====
105 // 画面描画
106 // =====
107
108 void draw() { // 毎フレームの描画関数
109   background(245); // 背景を明るい灰色で塗る
110   fill(0); // 文字色を黒に
111
112   text("Keyboard_test:", 30, 40); // 操作説明を表示
113   text("1:_Kick__2:_Cymbal", 30, 70); // キー割り当てを表示
114   text("A:_Auto_play_ON/OFF", 30, 100); // 自動演奏キーの説明
115   text("Arduino_serial:_1=_Kick,_2=_Cymbal", 30, 130); // シリアル入力の説明
116   text("Auto:_" + (autoPlay ? "ON" : "OFF"), 30, 170); // 自動演奏の状態を表示
117
118   if (autoPlay && millis() - lastTriggerTime >= 500) { // 自動演奏中で500ms経過した
      ら
119     lastTriggerTime = millis(); // 発音時刻を更新
120
121     if (autoStep == 0) { // ステップ0では
122       playKickSound(); // キックを鳴らす
123     } else if (autoStep == 1) { // ステップ1では
124       playCymbalSound(); // シンバルを鳴らす
125     } else if (autoStep == 2) { // ステップ2では
126       playKickSound(); // キックを鳴らす
127     } else if (autoStep == 3) { // ステップ3では
128       playCymbalSound(); // シンバルを鳴らす
129     }
130
131     autoStep = (autoStep + 1) % 4; // ステップを進める(4拍で1周)
132   }
133 }

```

```

134
135 // =====
136 // シリアル受信
137 // =====
138
139 void serialEvent(Serial port) { // シリアル受信時に呼ばれる関数
140     String receivedText = port.readStringUntil('\n'); // 1行読み取る
141
142     if (receivedText == null) return; // 空なら戻る
143
144     receivedText = trim(receivedText); // 前後の空白を除去
145     if (receivedText.length() == 0) return; // 空行なら戻る
146
147     println("recv:_" + receivedText); // 受信内容をログ表示
148
149     try { // 数値変換に備えて例外処理
150         int note = Integer.parseInt(receivedText); // 受信文字列を整数に変換
151
152         if (note == 1) { // 1なら
153             playKickSound(); // キックを鳴らす
154         } else if (note == 2) { // 2なら
155             playCymbalSound(); // シンバルを鳴らす
156         }
157     }
158     catch (NumberFormatException e) { // 数値でなければ
159         println("invalid_data:_" + receivedText); // 不正データとしてログ
160     }
161 }
162
163 // =====
164 // キー入力
165 // =====
166
167 void keyPressed() { // キー入力時に呼ばれる関数
168     if (key == '1') { // 1キーで
169         playKickSound(); // キックを鳴らす
170     } else if (key == '2') { // 2キーで
171         playCymbalSound(); // シンバルを鳴らす
172     } else if (key == 'a' || key == 'A') { // aキーで
173         autoPlay = !autoPlay; // 自動演奏をON/OFF切替
174         lastTriggerTime = millis(); // 発音時刻を初期化
175         autoStep = 0; // ステップを先頭に戻す
176     }
177 }
178
179 // =====
180 // キック音
181 // =====

```

```

182
183 void playKickSound() { // キック音を鳴らす関数
184     new Thread(new Runnable() { // 別スレッドでエンベロープを制御
185         public void run() { // スレッドの本体
186             final int durationMs = 650; // 発音の総時間(650ms)
187             final int updateMs = 2; // 音量を更新する周期(2ms)
188             int steps = durationMs / updateMs; // 更新回数
189
190             kickAmp.setValue(0.0); // 基音の音量を0に初期化
191             kickHarmonicAmp.setValue(0.0); // 倍音の音量を0に初期化
192
193             for (int i = 0; i < steps; i++) { // 各更新ステップについて
194                 float t = i * updateMs / 1000.0; // 経過時間(秒)
195
196                 float frequency = 48.0 + 122.0 * exp(-28.0 * t); // 高い音から48Hzへ急降下す
                    // 周波数
197                 float bodyEnvelope = exp(-8.0 * t); // 胴鳴りの減衰エンベロープ
198                 float attack = constrain(t / 0.004, 0.0, 1.0); // 4msの立上りを掛ける
199
200                 float bodyVolume = 0.82 * bodyEnvelope * attack; // 基音の音量を計算
201                 float harmonicVolume = 0.025 * exp(-18.0 * t) * attack; // 倍音の音量を計算
202
203                 kickOsc.setFrequency(frequency); // 基音の周波数を更新
204                 kickAmp.setValue(bodyVolume); // 基音の音量を設定
205
206                 kickHarmonic.setFrequency(frequency * 2.0); // 倍音を基音の2倍に設定
207                 kickHarmonicAmp.setValue(harmonicVolume); // 倍音の音量を設定
208
209                 sleepMs(updateMs); // 2ms待つ次の更新へ
210             }
211
212             kickAmp.setValue(0.0); // 最後に基音を無音化
213             kickHarmonicAmp.setValue(0.0); // 倍音も無音化
214         }
215     }).start(); // スレッドを開始
216 }
217
218 // =====
219 // シンバル音
220 // =====
221
222 void playCymbalSound() { // シンバル音を鳴らす関数
223     new Thread(new Runnable() { // 別スレッドでエンベロープを制御
224         public void run() { // スレッドの本体
225             final int durationMs = 700; // 発音の総時間(700ms)
226             final int updateMs = 4; // 音量を更新する周期(4ms)
227             int steps = durationMs / updateMs; // 更新回数
228

```

```

229     cymNoiseAmp.setValue(0.0); // ノイズ音量を0に初期化
230     cymMetalAmp1.setValue(0.0); // 金属倍音1を0に初期化
231     cymMetalAmp2.setValue(0.0); // 金属倍音2を0に初期化
232     cymMetalAmp3.setValue(0.0); // 金属倍音3を0に初期化
233
234     for (int i = 0; i < steps; i++) { // 各更新ステップについて
235         float t = i / float(steps - 1); // 0~1に正規化した進行度
236
237         float fastDecay = exp(-13.0 * t); // 速い減衰成分
238         float slowDecay = exp(-4.0 * t); // 遅い減衰成分
239
240         float noiseEnvelope = 0.25 * fastDecay + 0.10 * slowDecay; // ノイズのエンベ
           ロープ
241         float metalEnvelope = 0.09 * exp(-9.0 * t); // 金属倍音のエンベロープ
242         float attack = constrain(t * 45.0, 0.0, 1.0); // 急峻な立上りを掛ける
243
244         cymNoiseAmp.setValue(noiseEnvelope * attack); // ノイズの音量を設定
245         cymMetalAmp1.setValue(metalEnvelope * attack); // 金属倍音1の音量を設定
246         cymMetalAmp2.setValue(metalEnvelope * 0.75 * attack); // 金属倍音2の音量を設
           定
247         cymMetalAmp3.setValue(metalEnvelope * 0.50 * attack); // 金属倍音3の音量を設
           定
248
249         sleepMs(updateMs); // 4ms待つて次の更新へ
250     }
251
252     cymNoiseAmp.setValue(0.0); // 最後にノイズを無音化
253     cymMetalAmp1.setValue(0.0); // 金属倍音1を無音化
254     cymMetalAmp2.setValue(0.0); // 金属倍音2を無音化
255     cymMetalAmp3.setValue(0.0); // 金属倍音3を無音化
256 }
257 }).start(); // スレッドを開始
258 }
259
260 // =====
261 // 待機処理
262 // =====
263
264 void sleepMs(int milliseconds) { // 指定ミリ秒だけ待つ関数
265     try { // 割り込み例外に備える
266         Thread.sleep(milliseconds); // スレッドを一時停止
267     }
268     catch (InterruptedException e) { // 割り込まれたら
269         Thread.currentThread().interrupt(); // 割り込み状態を復元
270     }
271 }
272

```

```

273 // =====
274 // 終了処理
275 // =====
276
277 void stop() { // プログラム終了時の後始末関数
278     if (kickAmp != null) kickAmp.setValue(0.0); // キック基音を無音化
279     if (kickHarmonicAmp != null) kickHarmonicAmp.setValue(0.0); // キック倍音を無音化
280     if (cymNoiseAmp != null) cymNoiseAmp.setValue(0.0); // シンバルノイズを無音化
281     if (cymMetalAmp1 != null) cymMetalAmp1.setValue(0.0); // 金属倍音1を無音化
282     if (cymMetalAmp2 != null) cymMetalAmp2.setValue(0.0); // 金属倍音2を無音化
283     if (cymMetalAmp3 != null) cymMetalAmp3.setValue(0.0); // 金属倍音3を無音化
284
285     if (arduino != null) arduino.stop(); // シリアルポートを閉じる
286     if (out != null) out.close(); // 音声出力を閉じる
287     if (minim != null) minim.stop(); // Minimを停止
288
289     super.stop(); // 親クラスの終了処理を呼ぶ
290 }

```

G LCD 表示子機用 Arduino プログラム

LCD 表示に必要なプログラムを以下に示す。

```

1 #define DECODE_NEC // IRremoteでNECプロトコルのデコードを有効化
2 #include <IRremote.hpp> // 赤外線受信ライブラリを読み込む
3 #include <LiquidCrystal.h> // LCD制御ライブラリを読み込む
4
5 int childId = 0; // 自分の担当ビット番号
6 const int PIN_IR_RECV = 6; // 赤外線受光モジュールの入力ピン(D6)
7
8 // LCD 1台のみ (EN=11)
9 LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // LCDのピン接続を指定して初期化
10
11 // ---- カエルのカスタム文字 ----
12 // 左右共通 (char 0-2: 上段固定, char 3-5: 下段口パク)
13
14 byte frog0[8] = { // 左カエル上段左のドット絵(char0)
15     B00000, B00011, B00100, B01101, // 上4行のドットパターン
16     B01100, B00111, B00111, B01111 // 下4行のドットパターン
17 };
18 byte frog1_open[8] = { // 左カエル下段左・口開き(char3)
19     B11111, B11110, B11100, B11110, // 上4行のドットパターン
20     B01111, B00001, B00000, B00000 // 下4行のドットパターン
21 };
22 byte frog1_close[8] = { // 左カエル下段左・口閉じ(char3)
23     B11111, B11100, B11111, B11111, // 上4行のドットパターン
24     B01111, B00001, B00000, B00000 // 下4行のドットパターン

```



```

25 };
26 byte frog2[8] = { // カエル上段中央のドット絵(char1)
27     B00000, B10001, B01010, B01010, // 上4行のドットパターン
28     B01110, B11111, B11111, B10101 // 下4行のドットパターン
29 };
30 byte frog3_open[8] = { // カエル下段中央・口開き(char4)
31     B11111, B00000, B00000, B00000, // 上4行のドットパターン
32     B00000, B11111, B00000, B00000 // 下4行のドットパターン
33 };
34 byte frog3_close[8] = { // カエル下段中央・口閉じ(char4)
35     B11111, B00000, B11111, B11111, // 上4行のドットパターン
36     B11111, B11111, B00000, B00000 // 下4行のドットパターン
37 };
38 byte frog4[8] = { // カエル上段右のドット絵(char2)
39     B00000, B11000, B00100, B10110, // 上4行のドットパターン
40     B00110, B11100, B11100, B11110 // 下4行のドットパターン
41 };
42 byte frog5_open[8] = { // カエル下段右・口開き(char5)
43     B11111, B01111, B00111, B01111, // 上4行のドットパターン
44     B11110, B10000, B00000, B00000 // 下4行のドットパターン
45 };
46 byte frog5_close[8] = { // カエル下段右・口閉じ(char5)
47     B11111, B00111, B11111, B11111, // 上4行のドットパターン
48     B11110, B10000, B00000, B00000 // 下4行のドットパターン
49 };
50
51
52 // ---- 歌詞テーブル ----
53 #define TOTAL_TICKS 32 // 歌詞の総コマ数(1周32tick)
54
55 const char* lyrics[TOTAL_TICKS] = { // tickごとに表示する歌詞テーブル
56     "KA", // 0
57     "E", // 1
58     "RU", // 2
59     "NO", // 3
60     "U", // 4
61     "TA", // 5
62     "GA", // 6
63     "_", // 7
64     "KI", // 8
65     "KO", // 9
66     "E", // 10
67     "TE", // 11
68     "KU", // 12
69     "RU", // 13
70     "YO", // 14
71     "_", // 15
72     "GU", // 16

```

```

73  "WA",    // 17
74  "GU",    // 18
75  "WA",    // 19
76  "GU",    // 20
77  "WA",    // 21
78  "GU",    // 22
79  "WA",    // 23
80  "KERO",  // 24
81  "KERO",  // 25
82  "KERO",  // 26
83  "KERO",  // 27
84  "GWA!",  // 28
85  "GWA!",  // 29
86  "GWA!",  // 30
87  "_",     // 31
88  };
89
90
91  // ---- 状態変数 ----
92  long localPos = -1; // 現在の楽譜位置(-1は未受信)
93  bool frogOpen = false; // カエルの口の開閉状態
94
95  // 歌詞エリア: 列3-12 (幅×10) 2行
96  #define LYRIC_COL_START 3 // 歌詞表示エリアの開始列
97  #define LYRIC_WIDTH 10 // 歌詞表示エリアの幅(列数)
98
99  int displayCol = 0; // 現在の歌詞書き込み列
100 int displayRow = 0; // 現在の歌詞書き込み行
101
102
103  // ---- 関数 ----
104
105  // カスタム文字3-5を口の開閉に合わせて更新し、左右カエルを再描画
106  void updateFrogs(bool open) { // 口の開閉に合わせてカエルを再描画する関数
107      if (open) { // 口を開く場合
108          lcd.createChar(3, frog1_open); // char3を口開きに登録
109          lcd.createChar(4, frog3_open); // char4を口開きに登録
110          lcd.createChar(5, frog5_open); // char5を口開きに登録
111      } else { // 口を閉じる場合
112          lcd.createChar(3, frog1_close); // char3を口閉じに登録
113          lcd.createChar(4, frog3_close); // char4を口閉じに登録
114          lcd.createChar(5, frog5_close); // char5を口閉じに登録
115      }
116      // createChar後はカーソルをホームに戻してから書き直す
117      // 左カエル (列0)
118      lcd.setCursor(0, 0); // 左カエル上段の先頭へカーソル移動
119      lcd.write(byte(0)); lcd.write(byte(1)); lcd.write(byte(2)); // 上段3文字を描画
120      lcd.setCursor(0, 1); // 左カエル下段の先頭へ移動

```

```

121 lcd.write(byte(3)); lcd.write(byte(4)); lcd.write(byte(5)); // 下段3文字(口)を描画
122 // 右カエル (列13)
123 lcd.setCursor(13, 0); // 右カエル上段の先頭へ移動
124 lcd.write(byte(0)); lcd.write(byte(1)); lcd.write(byte(2)); // 上段3文字を描画
125 lcd.setCursor(13, 1); // 右カエル下段の先頭へ移動
126 lcd.write(byte(3)); lcd.write(byte(4)); lcd.write(byte(5)); // 下段3文字(口)を描画
127 }
128
129 // 歌詞エリア (列3-12) だけをスペースで埋めてクリア
130 void clearLyricArea() { // 歌詞エリアだけを消去する関数
131     for (int row = 0; row < 2; row++) { // 上下2行について
132         lcd.setCursor(LYRIC_COL_START, row); // 歌詞エリア先頭へ移動
133         for (int c = 0; c < LYRIC_WIDTH; c++) lcd.print(' '); // 幅の分だけ空白で埋める
134     }
135     displayCol = 0; // 書き込み列を先頭に戻す
136     displayRow = 0; // 書き込み行を先頭に戻す
137 }
138
139 void showLyric(int pos) { // 指定位置の歌詞を表示する関数
140     const char* word = lyrics[pos]; // 表示する語を取得
141     int len = strlen(word); // 語の文字数を求める
142
143     if (displayCol + len > LYRIC_WIDTH) { // 現在行に収まらなければ
144         if (displayRow == 0) { // まだ上段なら
145             displayRow = 1; // 下段へ改行
146             displayCol = 0; // 列を先頭に戻す
147         } else { // すでに下段なら
148             clearLyricArea(); // エリアを消して先頭から書き直す
149         }
150     }
151
152     lcd.setCursor(LYRIC_COL_START + displayCol, displayRow); // 書き込み位置へカーソル
        移動
153     lcd.print(word); // 歌詞を表示
154     displayCol += len; // 次の書き込み列へ進める
155 }
156
157
158 void setup() { // 起動時の初期化関数
159     Serial.begin(115200); // シリアル通信を開始
160     lcd.begin(16, 2); // 16桁2行のLCDを初期化
161
162     lcd.createChar(0, frog0); // char0にカエル絵を登録
163     lcd.createChar(1, frog2); // char1にカエル絵を登録
164     lcd.createChar(2, frog4); // char2にカエル絵を登録
165     lcd.createChar(3, frog1_close); // char3を口閉じで登録
166     lcd.createChar(4, frog3_close); // char4を口閉じで登録
167     lcd.createChar(5, frog5_close); // char5を口閉じで登録

```

```

168
169 lcd.clear(); // 画面を消去
170 updateFrogs(false); // 口を閉じた状態でカエルを描画
171
172 IrReceiver.begin(PIN_IR_RECV); // 赤外線受信をD6で開始
173 Serial.println("setup_OK"); // 初期化完了をログ出力
174 }
175
176 void loop() { // メインループ
177     if (!IrReceiver.decode()) return; // IRを受信していなければ戻る
178     IrReceiver.printIRResultShort(&Serial); // 受信内容を簡易表示
179
180     if (IrReceiver.decodedIRData.protocol == NEC) { // NEC信号のときだけ処理
181         uint8_t mask = IrReceiver.decodedIRData.command; // コマンド(参加マスク)を取り出す
182
183         if (mask & (1 << childId)) { // 自分の担当ビットが立っていれば
184             localPos++; // 楽譜位置を1進める
185             if (localPos >= TOTAL_TICKS) { // 末尾を超えたら
186                 localPos = 0; // 先頭へ戻す
187                 clearLyricArea(); // 歌詞エリアを消去
188             }
189
190             showLyric((int)localPos); // 現在位置の歌詞を表示
191             Serial.println("kasiOK"); // 歌詞表示完了をログ出力
192             delay(10); // 10ms待つ
193
194             frogOpen = !frogOpen; // 口の開閉を反転
195             updateFrogs(frogOpen); // カエルを再描画(口パク)
196             Serial.println("kaoOK"); // 顔更新完了をログ出力
197             delay(10); // 10ms待つ
198         }
199     }
200
201     IrReceiver.resume(); // 次の受信に備えて再開
202 }

```